

Lead Management System - Architecture v2.0

Table of Contents

Executive Summary

Vision

Key Objectives

Current State → Future State

System Overview

High-Level Architecture

Architecture Principles

- 1. Multi-Tenancy
- 2. Immutability
- 3. Separation of Concerns
- 4. Eventual Consistency
- 5. Configuration as Data

Data Models & Schema

DynamoDB Tables

S3 Data Structure

Lead Processing Flow

End-to-End Flow Diagram

Detailed Flow Steps

API Design

Base URL

Authentication

Endpoints

Management APIs

Webhook Formats

Core Services

Service Architecture

- 1. Mapping Service
- 2. Quality Check Service
- 3. Routing Service
- 4. Webhook Service

Quality Checks & Plugins

Plugin System Architecture

Plugin Configuration Schema

IPQS Plugin Configuration

TrustedForm Plugin Configuration

Duplicate Check Plugin Configuration

Account-Level Plugin Credentials

Routing & Distribution

Routing Types

Lead Caps

Webhooks & Notifications

Webhook Delivery Architecture

Client Webhook Delivery	
Webhook Security	
Affiliate Postback Delivery	
Email Notifications	
AWS Infrastructure	
Architecture Diagram	
CDK Stack Structure	
Resource Naming Convention	
Lambda Functions	
DynamoDB Tables	
S3 Buckets	
SQS Queues	
Secrets Manager	
Migration Strategy	
Phase 1: Data Migration (Week 1-2)	
Phase 2: Affiliate & Client Setup (Week 3-4)	
Phase 3: Parallel Run (Week 5-6)	
Phase 4: Cutover (Week 7)	
Rollback Plan	
Appendices	
A. Unique ID Generation	
B. Testing Configuration Examples	
C. Front-End Integration Considerations	
D. Performance Estimates	
E. Cost Estimation (Monthly)	
Conclusion	

Lead Management System - Architecture v2.0

Version: 2.0
Date: February 4, 2026
Status: Design Document
Author: Architecture Team

Table of Contents

- [Executive Summary](#)
- [System Overview](#)
- [Architecture Principles](#)
- [Data Models & Schema](#)
- [Lead Processing Flow](#)
- [API Design](#)
- [Core Services](#)
- [Quality Checks & Plugins](#)
- [Routing & Distribution](#)
- [Webhooks & Notifications](#)
- [AWS Infrastructure](#)
- [Migration Strategy](#)

Executive Summary

Vision

Transform the current prototype lead intake system into a production-ready, multi-tenant SaaS platform with configurable campaign management, flexible lead routing, and extensible plugin architecture.

Key Objectives

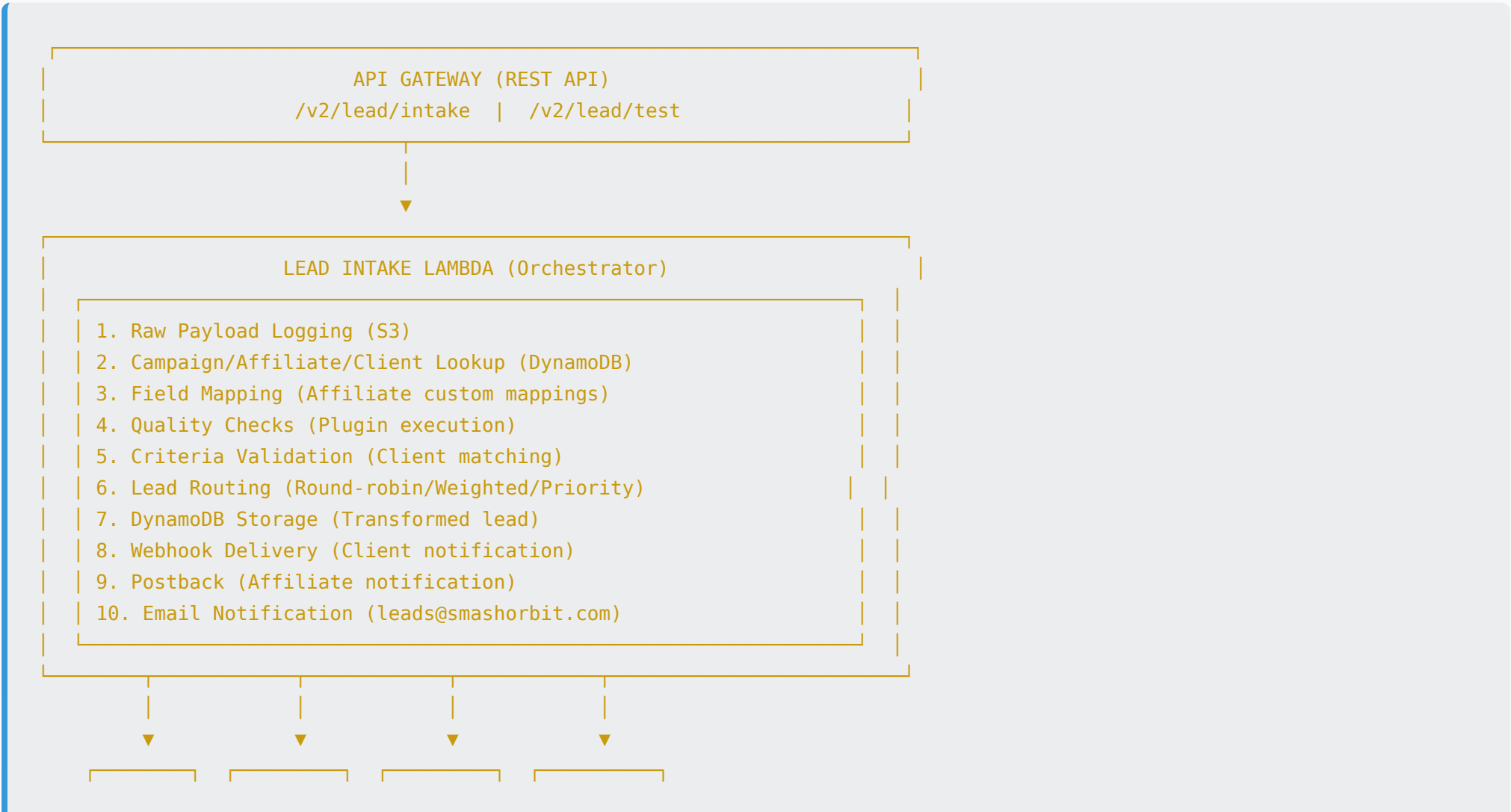
- Data Integrity:** Store raw lead payloads in S3 Parquet for audit trails and analytics
- Flexibility:** Support custom field mappings per affiliate and custom criteria per client
- Scalability:** Handle versioning for campaign configurations without breaking existing flows
- Extensibility:** Plugin-based architecture for quality checks (IPQS, TrustedForm, Duplicate Detection)
- Reliability:** Webhook delivery with retry mechanisms and comprehensive notifications

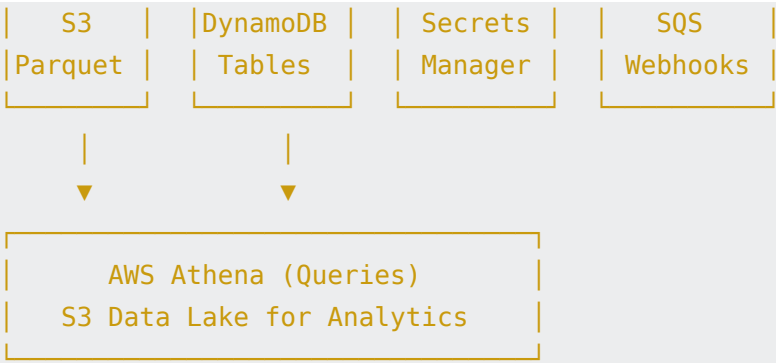
Current State → Future State

Aspect	Current (v1.0 Prototype)	Future (v2.0 Production)
Architecture	Single prototype API	Multi-tenant SaaS platform
Configuration	Hardcoded validation rules	Campaign-based configurable rules
Data Storage	DynamoDB only	S3 Parquet (raw) + DynamoDB (transformed)
Affiliates	Not supported	Full affiliate management with custom mappings
Clients	Not supported	Client management with custom criteria
Routing	None	Round-robin, weighted, priority, traffic-based
Plugins	Hardcoded IPQS + TrustedForm	Extensible plugin system
Webhooks	None	Client webhooks + affiliate postbacks
Versioning	None	Campaign criteria/logic versioning
Testing	Production API only	Dedicated test mode with configurable bypass

System Overview

High-Level Architecture





Architecture Principles

1. Multi-Tenancy

- Each tenant (account) is isolated with separate credentials and configurations
- Shared infrastructure with tenant-level data segregation
- Tenant identifier prefix in all resource names

2. Immutability

- Raw lead payloads never modified in S3
- Versioned configurations maintain historical context
- Audit trail for all configuration changes

3. Separation of Concerns

- Quality checks as independent, pluggable services
- Campaign configuration separate from lead processing logic
- Clear service boundaries (mapping, validation, routing, delivery)

4. Eventual Consistency

- Webhooks delivered asynchronously via SQS
- Lead processing doesn't block on webhook delivery
- Retry mechanisms for failed webhooks

5. Configuration as Data

- All rules stored as JSON in DynamoDB catalogs
- No hardcoded business logic
- Version-controlled configuration changes

Data Models & Schema

DynamoDB Tables

1. Leads Table

Table Name: `{tenant}-{system}-{env}-table-leads`

Purpose: Store transformed, processed leads

```
{
  // Primary Key
  "id": "LEAD12345678", // PK: Lead unique ID (auto-generated)
  "timestamp": "2026-02-04T12:30:45.123Z", // SK: ISO timestamp

  // Lead Data (transformed)
  "campaign_id": "CAMP12345678",
  "affiliate_id": "AFF12345678",
  "client_id": "CLI12345678", // Assigned client after routing
  "created_at": "2026-02-04T12:30:45.123Z",
  "date": "2026-02-04",
  "time": "12:30:45",

  // Contact Information
```

```
"first_name": "John",
"last_name": "Doe",
"email": "john@example.com",
"phone": "1234567890",
"state": "TX",
"ip_address": "1.2.3.4",

// Campaign Data
"campaign_name": "Rideshare Campaign Q1",
"affiliate_name": "Partner ABC",
"client_name": "Law Firm XYZ",

// S3 Reference
"s3_raw_payload_path": "s3://bucket/leads/2026/02/04/LEAD12345678.parquet",
"remapped": true, // Boolean: Was affiliate custom mapping applied?

// Quality Check Results
"quality_checks": {
  "ipqs": {
    "enabled": true,
    "passed": true,
    "results": { ... }
  },
  "trustedform": {
    "enabled": true,
    "passed": true,
    "results": { ... }
  },
  "duplicate": {
    "enabled": true,
    "passed": true,
    "duplicate_found": false
  }
},

// Criteria Validation
"criteria_passed": true,
"criteria_version": 1,
"logic_passed": true,
"logic_version": 1,

// Rejection Tracking (Core Principle: No lead is ever dropped)
"rejected": false, // Master rejection flag
"rejection_flags": { // Specific rejection reasons
  "invalid_campaign_key": false,
  "inactive_campaign": false,
  "test_affiliate_on_prod": false,
  "live_affiliate_on_test": false,
  "inactive_affiliate": false,
  "cap_exceeded": false,
  "duplicate": false,
  "ipqs_failed": false,
  "trustedform_failed": false,
  "criteria_failed": false,
  "logic_failed": false
},
"rejection_reason": null, // Human-readable primary reason (from campaign config)
"rejection_details": {}, // Additional details about rejection
"validation_skipped": false, // If true, IPQS/TF checks were skipped due to early rejection

// Routing & Status
"routing_rule": "round_robin",
"sellable": true,
"sold": true,
"sold_at": "2026-02-04T12:30:50.123Z",
"sale_price": 50.00,

// Cherry Pick Configuration
"cherry_pick_enabled": true, // If true, this lead can be cherry picked
"cherry_picked": false, // If true, lead has been manually selected/assigned
"cherry_picked_by": null, // User ID who cherry picked this lead
"cherry_picked_at": null, // Timestamp when cherry picked
```

```
// Edit Tracking
"edited": true, // Flag indicating if lead has been edited after creation
"last_edited_by": "user@smashorbit.com",
"last_edited_at": "2026-02-05T14:30:00.000Z",
"edit_count": 2, // Number of times lead has been edited
"edited_fields": ["phone", "email"], // Array of field names that have been modified

// Webhook & Notifications
"webhook_sent": true,
"webhook_response_code": 200,
"webhook_sent_at": "2026-02-04T12:30:51.123Z",
"postback_sent": true,
"postback_sent_at": "2026-02-04T12:30:52.123Z",
"email_sent": true,

// Additional Fields (from affiliate payload)
"rideshare_abuse": true,
"rideshare_company": "Uber",
"utm_source": "google",
"utm_campaign": "rideshare-q1",

// Metadata
"test_mode": false,
"tenant_id": "smashorbit",
"version": "2.0"
}
```

Global Secondary Indexes: - `campaign-index` : PK: `campaign_id` , SK: `timestamp` - `affiliate-index` : PK: `affiliate_id` , SK: `timestamp` - `client-index` : PK: `client_id` , SK: `timestamp` - `phone-index` : PK: `phone` , SK: `timestamp` (for duplicate detection) - `email-index` : PK: `email` , SK: `timestamp` (for duplicate detection) - `date-index` : PK: `date` , SK: `timestamp` (for daily reports)

2. Campaigns Table

Table Name: `{tenant}-{system}-{env}-table-campaigns`

Purpose: Campaign definitions and configurations

```
{
  // Primary Key
  "id": "CAMP12345678", // PK: Campaign unique ID
  "created_at": "2026-02-01T10:00:00.000Z", // SK: Creation timestamp

  // Campaign Details
  "name": "Rideshare Abuse Campaign Q1 2026",
  "status": "active", // active | inactive
  "tenant_id": "smashorbit",

  // Versioning
  "current_criteria_version": 1,
  "current_logic_version": 1,

  // Affiliates (linked to this campaign)
  "affiliates": [
    {
      "affiliate_id": "AFF12345678",
      "campaign_key": "CAMPKEY_ABC123XYZ789", // Unique API key for this affiliate-campaign relationship
      "status": "test", // test | live | inactive | deleted (defaults to 'test' when added)
      "added_at": "2026-02-01T10:30:00.000Z",
      "status_changed_at": "2026-02-01T10:30:00.000Z",
      "previous_status": null,

      // Custom Mapping Configuration
      "custom_mapping_enabled": true,
      "field_mappings": {
        "fname": "first_name", // affiliate_field -> campaign_field (key mapping)
        "lname": "last_name",
        "contact_email": "email",
        "phone_num": "phone"
      }
    },
  ],
}
```

```

"value_mappings": {                                // Value-level transformations
  "assault_option": {                              // Field name
    "rape": "Rape / groping",                     // affiliate_value -> canonical_value
    "assault": "Physical assault",
    "harassment": "Sexual harassment"
  },
  "rideshare_company": {
    "uber": "Uber",
    "lyft": "Lyft",
    "other": "Other"
  }
},

// Lead Caps & Tracking
"cap_type": "daily",                             // daily | weekly | monthly | overall
"cap_limit": 100,                                // Maximum leads allowed for the cap period
"leads_sent_current_period": 45,                 // Counter for current period
"cap_period_start": "2026-02-04",                // Start date of current cap period (UTC)
"cap_period_end": "2026-02-05",                  // End date of current cap period (UTC)
"total_leads_sent": 1250,                        // All-time counter (for overall cap type)

// Postback Configuration
"postback_url": "https://affiliate.com/postback",
"postback_enabled": true
},

],

// Clients (linked to this campaign)
"clients": [
  {
    "client_id": "CLI12345678",
    "status": "active",
    "added_at": "2026-02-01T11:00:00.000Z",
    "status_changed_at": "2026-02-01T11:00:00.000Z",
    "previous_status": null,

    // Criteria/Logic Configuration
    "criteria_config": {
      "use_custom": false,                        // If false, uses campaign's current_criteria_version
      "criteria_version": null                    // null = use campaign default, or specify version
    },
    "logic_config": {
      "use_custom": false,                        // If false, uses campaign's current_logic_version
      "logic_version": null                       // null = use campaign default, or specify version
    },

    // Routing Configuration
    "routing_weight": 50,                        // For weighted routing (percentage or ratio)
    "routing_priority": 1,                       // For priority routing (1=highest)

    // Webhook Configuration
    "webhook_url": "https://client.com/leads",
    "webhook_enabled": true,
    "webhook_retry_attempts": 3
  }
],

// Routing Configuration
"routing_type": "round_robin",                   // round_robin | weighted | priority | traffic_based
"routing_state": {
  "last_client_index": 0,                        // For round-robin
  "client_counts": {}                            // For tracking distribution
},

// Quality Checks (Plugins)
"plugins": {
  "ipqs": {
    "enabled": true,
    "config": {
      "phone_enabled": true,
      "email_enabled": true,
      "ip_enabled": true,

```

```

    "thresholds": {
      "phone": {
        "fraud_score": { "operator": "<", "value": 75 },
        "VOIP": { "operator": "==", "value": false },
        "valid": { "operator": "==", "value": true }
      },
      "email": {
        "fraud_score": { "operator": "<", "value": 75 },
        "valid": { "operator": "==", "value": true },
        "disposable": { "operator": "==", "value": false }
      },
      "ip": {
        "fraud_score": { "operator": "<", "value": 75 },
        "proxy": { "operator": "==", "value": false },
        "vpn": { "operator": "==", "value": false }
      }
    },
    "trustedform": {
      "enabled": true,
      "config": {
        "require_valid_cert": true
      }
    },
    "duplicate_check": {
      "enabled": true,
      "config": {
        "phone_enabled": true,
        "email_enabled": true,
        "operator": "OR", // OR = reject if phone OR email exists
                        // AND = reject only if BOTH exist
        "action": "reject" // reject | flag | allow
      }
    },
    // Test Mode Configuration
    "test_config": {
      "bypass_quality_checks": false,
      "bypass_criteria": false,
      "bypass_duplicate_check": true,
      "skip_webhooks": true,
      "skip_emails": true
    },
    // Lead Edit Configuration
    "lead_edit_config": {
      "enabled": true, // Allow editing leads in this campaign
      "require_reason": true, // Require reason for edits
      "editable_fields": [ // Fields that can be edited
        "first_name",
        "last_name",
        "email",
        "phone",
        "state",
        "rideshare_company",
        "has_attorney"
      ],
      "readonly_fields": [ // Fields that cannot be edited
        "id",
        "timestamp",
        "campaign_id",
        "affiliate_id",
        "client_id",
        "created_at",
        "ip_address",
        "trusted_form_cert",
        "quality_checks"
      ]
    },

```

```
// Rejection Message Configuration
"rejection_messages": {
  "invalid_campaign_key": "Invalid campaign key provided",
  "inactive_campaign": "Lead received after campaign was marked inactive",
  "test_affiliate_on_prod": "Test affiliate attempted to use production endpoint",
  "live_affiliate_on_test": "Live affiliate attempted to use test endpoint",
  "inactive_affiliate": "Lead received from inactive affiliate",
  "cap_exceeded": "Affiliate daily lead cap exceeded",
  "duplicate": "Duplicate lead detected (phone or email already exists)",
  "ipqs_failed": "Lead failed IPQS quality checks",
  "trustedform_failed": "Lead failed TrustedForm certificate validation",
  "criteria_failed": "Lead missing required campaign criteria fields",
  "logic_failed": "Lead did not meet campaign qualification logic"
},

// Metadata
"updated_at": "2026-02-04T12:00:00.000Z",
"created_by": "user@smashorbit.com"
}
```

Global Secondary Indexes: - `status-index` : PK: `status` , SK: `created_at` - `tenant-index` : PK: `tenant_id` , SK: `created_at` - `campaign-key-index` : PK: `campaign_key` (sparse index - only on affiliate campaign_key values in affiliates array)

Note: The `campaign-key-index` is a sparse GSI that indexes each unique `campaign_key` from the nested `affiliates[]` array. This allows fast lookups of campaigns by the campaign_key provided in lead submissions without requiring both campaign_id and affiliate_id.

3. Affiliates Table

Table Name: `{tenant}-{system}-{env}-table-affiliates`
Purpose: Affiliate master data

```
{
  // Primary Key
  "id": "AFF12345678", // PK: Affiliate unique ID
  "created_at": "2026-01-15T09:00:00.000Z", // SK: Creation timestamp

  // Affiliate Details
  "name": "Partner ABC Marketing",
  "code": "ABC123", // Unique code for cross-system identification
  "status": "active", // active | inactive
  "tenant_id": "smashorbit",

  // Contact Information
  "contact_email": "partner@abc.com",
  "contact_phone": "5551234567",

  // Campaign Associations (denormalized for quick lookup)
  "campaigns": [
    "CAMP12345678",
    "CAMP87654321"
  ],

  // Statistics (updated periodically)
  "total_leads_sent": 1250,
  "total_leads_accepted": 980,
  "acceptance_rate": 78.4,

  // Metadata
  "updated_at": "2026-02-04T12:00:00.000Z",
  "created_by": "admin@smashorbit.com"
}
```

Global Secondary Indexes: - `code-index` : PK: `code` , SK: `created_at` (unique code lookup) - `status-index` : PK: `status` , SK: `created_at` - `tenant-index` : PK: `tenant_id` , SK: `created_at`

4. Clients Table

Table Name: {tenant}-{system}-{env}-table-clients

Purpose: Client master data

```
{
  // Primary Key
  "id": "CLI12345678", // PK: Client unique ID
  "created_at": "2026-01-20T10:00:00.000Z", // SK: Creation timestamp

  // Client Details
  "name": "Smith & Associates Law Firm",
  "code": "SMITH001", // Unique code for cross-system identification
  "status": "active", // active | inactive
  "tenant_id": "smashorbit",

  // Contact Information
  "contact_email": "leads@smithlaw.com",
  "contact_phone": "5559876543",

  // Campaign Associations
  "campaigns": [
    "CAMP12345678"
  ],

  // Statistics
  "total_leads_received": 450,
  "total_leads_purchased": 420,
  "purchase_rate": 93.3,

  // Metadata
  "updated_at": "2026-02-04T12:00:00.000Z",
  "created_by": "admin@smashorbit.com"
}
```

Global Secondary Indexes: - code-index : PK: code , SK: created_at - status-index : PK: status , SK: created_at - tenant-index : PK: tenant_id , SK: created_at

5. Users Table

Table Name: {tenant}-{system}-{env}-table-users

Purpose: User authentication and authorization for dashboard access

```
{
  // Primary Key
  "id": "USER12345678", // PK: User unique ID
  "email": "user@smashorbit.com", // SK: User email (unique)

  // User Details
  "first_name": "John",
  "last_name": "Doe",
  "display_name": "John Doe",
  "status": "active", // active | inactive | suspended
  "tenant_id": "smashorbit",

  // Authentication
  "password_hash": "$2b$10$...", // Bcrypt hashed password
  "mfa_enabled": false,
  "mfa_secret": null,

  // Authorization
  "role": "admin", // admin | manager | viewer | editor
  "permissions": [
    "leads.view",
    "leads.edit",
    "leads.cherry_pick",
    "campaigns.manage",
    "affiliates.manage",
    "clients.manage",
    "users.manage"
  ]
}
```

```
],

// Session Management
"last_login_at": "2026-02-05T10:00:00.000Z",
"last_login_ip": "1.2.3.4",
"session_token": "jwt_token_here",
"token_expires_at": "2026-02-06T10:00:00.000Z",

// Metadata
"created_at": "2026-01-15T10:00:00.000Z",
"updated_at": "2026-02-05T10:00:00.000Z",
"created_by": "admin@smashorbit.com"
}
```

Global Secondary Indexes: - email-index : PK: email , SK: created_at - tenant-role-index : PK: tenant_id , SK: role - status-index : PK: status , SK: last_login_at

6. Templates Table

Table Name: {tenant}-{system}-{env}-table-templates
Purpose: Store reusable template configurations for criteria and logic that can be used across campaigns

```
{
  // Primary Key
  "id": "TPL12345678",           // PK: Template unique ID
  "type": "criteria",           // SK: criteria | logic

  // Template Details
  "name": "Standard Rideshare Intake",
  "description": "Standard questions for rideshare abuse cases",
  "category": "rideshare",       // For organizing templates (rideshare, personal_injury, etc.)
  "status": "active",           // active | archived
  "is_system_template": true,    // System templates vs custom tenant templates
  "tenant_id": "system",        // 'system' for built-in templates, tenant_id for custom

  // Configuration (same structure as Config Catalog)
  "config": {
    "questions": [ /* criteria questions */ ]
    // OR
    "rules": [ /* logic rules */ ]
  },

  // Usage Statistics
  "times_used": 25,
  "campaigns_using": 5,

  // Metadata
  "created_at": "2026-01-15T10:00:00.000Z",
  "updated_at": "2026-02-01T12:00:00.000Z",
  "created_by": "admin@smashorbit.com"
}
```

Global Secondary Indexes: - type-index : PK: type , SK: created_at - category-index : PK: category , SK: created_at - tenant-index : PK: tenant_id , SK: created_at

7. Lead Edit History Table

Table Name: {tenant}-{system}-{env}-table-lead-edits
Purpose: Audit trail of all lead field modifications

```
{
  // Primary Key
  "lead_id": "LEAD12345678",     // PK: Lead being edited
  "edit_timestamp": "2026-02-05T14:30:00.000Z", // SK: When the edit occurred

  // Edit Details
```

```
"edit_id": "EDIT12345678",           // Unique edit ID
"edited_by": "user@smashorbit.com",   // User who made the change
"edited_by_name": "John Doe",
"edit_type": "field_update",         // field_update | cherry_pick | status_change

// Changed Fields
"changes": [
  {
    "field_name": "phone",
    "field_display_name": "Phone Number",
    "old_value": "5551234567",
    "new_value": "5559876543",
    "change_reason": "Customer requested update"
  },
  {
    "field_name": "email",
    "field_display_name": "Email Address",
    "old_value": "old@example.com",
    "new_value": "new@example.com",
    "change_reason": "Typo correction"
  }
],

// Context
"ip_address": "1.2.3.4",
"user_agent": "Mozilla/5.0...",
"session_id": "session_abc123",

// Metadata
"tenant_id": "smashorbit",
"campaign_id": "CAMP12345678"
}
```

Global Secondary Indexes: - `user-index` : PK: `edited_by` , SK: `edit_timestamp` - `campaign-index` : PK: `campaign_id` , SK: `edit_timestamp` - `edit-type-index` : PK: `edit_type` , SK: `edit_timestamp`

8. Configuration Catalog Table

Table Name: `{tenant}-{system}-{env}-table-config-catalog`
Purpose: Store versioned campaign criteria and logic configurations

```
{
  // Primary Key
  "campaign_id": "CAMP12345678",           // PK: Campaign ID
  "config_type": "criteria#v1",           // SK: Type and version (criteria#v1, logic#v2)

  // Configuration Details
  "version": 1,
  "status": "active",                     // active | archived
  "created_at": "2026-02-01T10:00:00.000Z",
  "created_by": "user@smashorbit.com",

  // Criteria Questions Configuration
  "config": {
    "questions": [
      {
        "key": "rideshare_abuse",
        "display_name": "Did you experience rideshare abuse?",
        "data_type": "boolean",
        "description": "Indicates if the lead experienced abuse from rideshare services",
        "required": true
      },
      {
        "key": "rideshare_company",
        "display_name": "Which rideshare company?",
        "data_type": "list",
        "description": "The rideshare company involved",
        "required": true,
        "options": [
```

```

        { "label": "Uber", "value": "uber" },
        { "label": "Lyft", "value": "lyft" },
        { "label": "Other", "value": "other" }
    ]
},
{
    "key": "state",
    "display_name": "State",
    "data_type": "string",
    "description": "State where incident occurred",
    "required": true
},
{
    "key": "has_attorney",
    "display_name": "Do you have an attorney?",
    "data_type": "boolean",
    "description": "Whether the lead already has legal representation",
    "required": false
}
]
},
// Used By (tracking which clients/campaigns use this version)
"used_by": [
    {
        "type": "campaign_base",
        "id": "CAMP12345678"
    },
    {
        "type": "client_custom",
        "campaign_id": "CAMP12345678",
        "client_id": "CLI87654321"
    }
]
}

```

Example Logic Configuration:

```

{
    "campaign_id": "CAMP12345678",
    "config_type": "logic#v1",
    "version": 1,
    "status": "active",
    "created_at": "2026-02-01T10:15:00.000Z",

    // Logic Validation Rules
    "config": {
        "rules": [
            {
                "name": "Rule 1: Rideshare abuse with no attorney",
                "groups": [
                    {
                        "operator": "AND",
                        "conditions": [
                            {
                                "field": "rideshare_abuse",
                                "operator": "==",
                                "value": true
                            },
                            {
                                "field": "has_attorney",
                                "operator": "==",
                                "value": false
                            }
                        ]
                    }
                ]
            }
        ],
        "action": "accept"
    },
}

```

```
{
  "name": "Rule 2: Uber or Lyft only",
  "groups": [
    {
      "operator": "OR",
      "conditions": [
        {
          "field": "rideshare_company",
          "operator": "in",
          "value": ["uber", "lyft"]
        }
      ]
    }
  ],
  "action": "accept"
},
"combine_rules": "AND"           // AND = all rules must pass, OR = any rule passes
}
```

Global Secondary Indexes: - `version-index` : PK: `campaign_id` , SK: `version` (ascending) - `status-index` : PK: `status` , SK: `created_at`

6. Tenant Configuration Table

Table Name: `{tenant}-{system}-{env}-table-tenant-config`

Purpose: Store account-level configurations and credentials

```
{
  // Primary Key
  "tenant_id": "smashorbit",           // PK: Tenant identifier
  "config_type": "account_settings",   // SK: Config type

  // Account Information
  "account_name": "SmashOrbit",
  "status": "active",
  "created_at": "2026-01-01T00:00:00.000Z",

  // Plugin Credentials (references to Secrets Manager)
  "plugin_credentials": {
    "ipqs": {
      "configured": true,
      "secret_arn": "arn:aws:secretsmanager:us-east-1:123456789012:secret:smashorbit/ipqs",
      "configured_at": "2026-01-15T12:00:00.000Z"
    },
    "trustedform": {
      "configured": true,
      "secret_arn": "arn:aws:secretsmanager:us-east-1:123456789012:secret:smashorbit/trustedform",
      "configured_at": "2026-01-15T12:05:00.000Z"
    }
  },

  // Email Configuration
  "notification_emails": {
    "leads": "leads@smashorbit.com",
    "alerts": "alerts@smashorbit.com",
    "admin": "admin@smashorbit.com"
  },

  // Cherry Pick Configuration (Global)
  "cherry_pick_config": {
    "enabled": true,                  // Enable cherry pick feature system-wide
    "default_enabled_for_new_leads": false, // Default cherry_pick_enabled value for new leads
    "roles_allowed_to_cherry_pick": [   // User roles that can cherry pick leads
      "admin",
      "manager"
    ]
  },
}
```

```
// User Roles & Permissions (Global)
"user_permissions": {
  "roles_allowed_to_edit_leads": [    // User roles that can edit leads (if campaign allows)
    "admin",
    "manager",
    "editor"
  ]
},

// System Settings
"settings": {
  "default_timezone": "America/Chicago",
  "default_currency": "USD",
  "lead_retention_days": 365
}
}
```

S3 Data Structure

Raw Lead Payloads (Parquet Format)

Bucket Structure:

```
s3://{tenant}-{system}-{env}-leads-raw/
├─ leads/
│   └─ 2026/
│       └─ 01/
│           └─ 01/
│               └─ leads-2026-01-01.parquet
│           └─ 02/
│               └─ leads-2026-01-02.parquet
│       └─ 02/
│           └─ 01/
│               └─ leads-2026-02-01.parquet
│           └─ 04/
│               └─ leads-2026-02-04.parquet (current day, appended)
```

Parquet Schema:

```
{
  "lead_id": "LEAD12345678",
  "timestamp": "2026-02-04T12:30:45.123Z",
  "campaign_id": "CAMP12345678",
  "affiliate_id": "AFF12345678",

  // Raw payload as received from affiliate
  "raw_payload": {
    "fname": "John",                // Original field name from affiliate
    "lname": "Doe",
    "contact_email": "john@example.com",
    "phone_num": "1234567890",
    "rideshare_company_value": "uber",
    "ip": "1.2.3.4"
  },

  // Field mapping applied (if custom mapping enabled)
  "field_mappings": {
    "fname": "first_name",
    "lname": "last_name",
    "contact_email": "email",
    "phone_num": "phone",
    "rideshare_company_value": "rideshare_company",
    "ip": "ip_address"
  },

  // Transformed payload (stored in DynamoDB)
  "transformed_payload": {
    "first_name": "John",
```

```
        "last_name": "Doe",
        "email": "john@example.com",
        "phone": "1234567890",
        "rideshare_company": "uber",
        "ip_address": "1.2.3.4"
    },

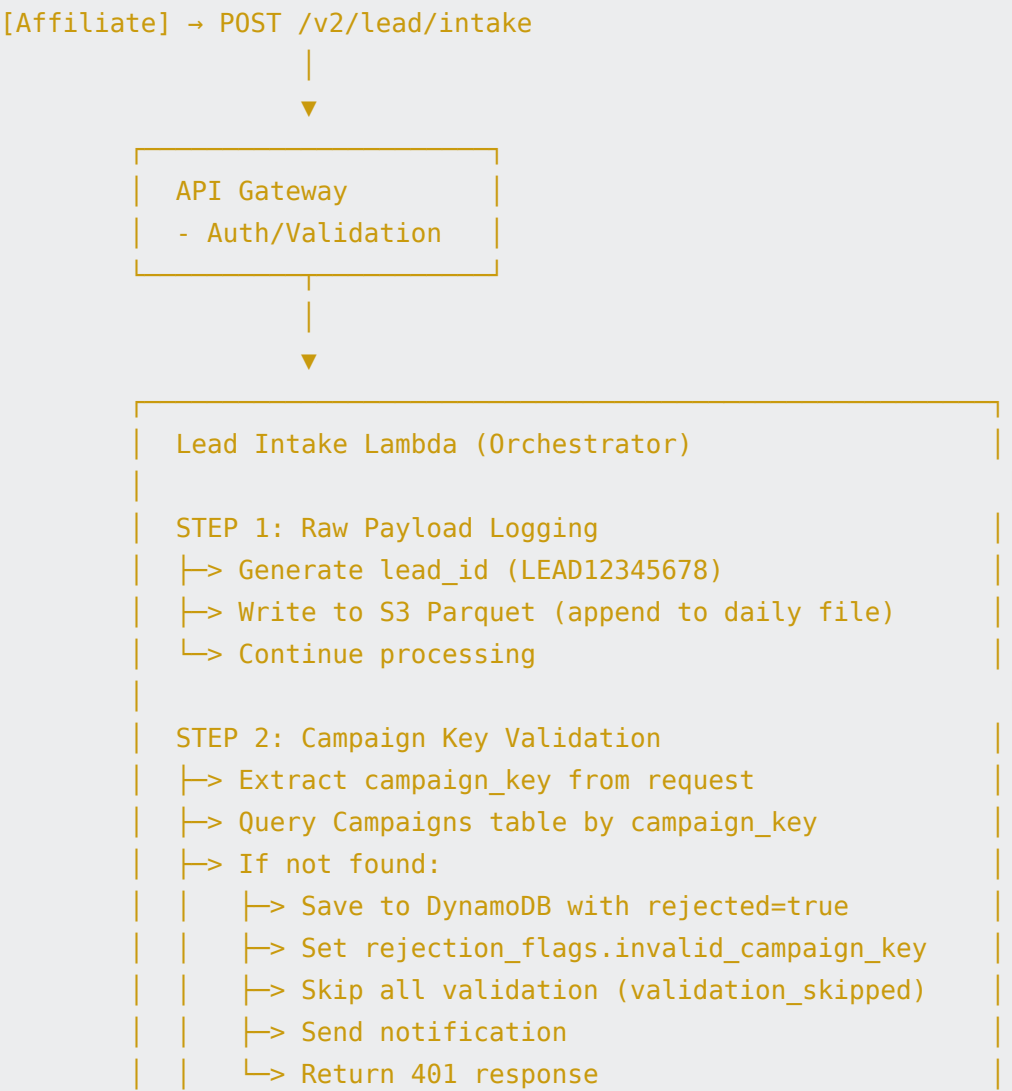
    // Processing metadata
    "remapped": true,
    "tenant_id": "smashorbit"
}
```

Athena Table Definition:

```
CREATE EXTERNAL TABLE IF NOT EXISTS leads_raw (
    lead_id STRING,
    timestamp TIMESTAMP,
    campaign_id STRING,
    affiliate_id STRING,
    raw_payload STRUCT<
        fname: STRING,
        lname: STRING,
        contact_email: STRING,
        phone_num: STRING,
        rideshare_company_value: STRING,
        ip: STRING
    >,
    field_mappings MAP<STRING, STRING>,
    transformed_payload MAP<STRING, STRING>,
    remapped BOOLEAN,
    tenant_id STRING
)
PARTITIONED BY (year INT, month INT, day INT)
STORED AS PARQUET
LOCATION 's3://{bucket}/leads/';
```

Lead Processing Flow

End-to-End Flow Diagram



```

└─> If found → Continue to step 3

STEP 3: Campaign Status Check
└─> Check campaign.status
└─> If status != "active":
|   └─> Save to DynamoDB with rejected=true
|   └─> Set rejection_flags.inactive_campaign
|   └─> Skip all validation (validation_skipped)
|   └─> Send notification (especially in prod)
|   └─> Return 403 response
└─> If active → Continue to step 4

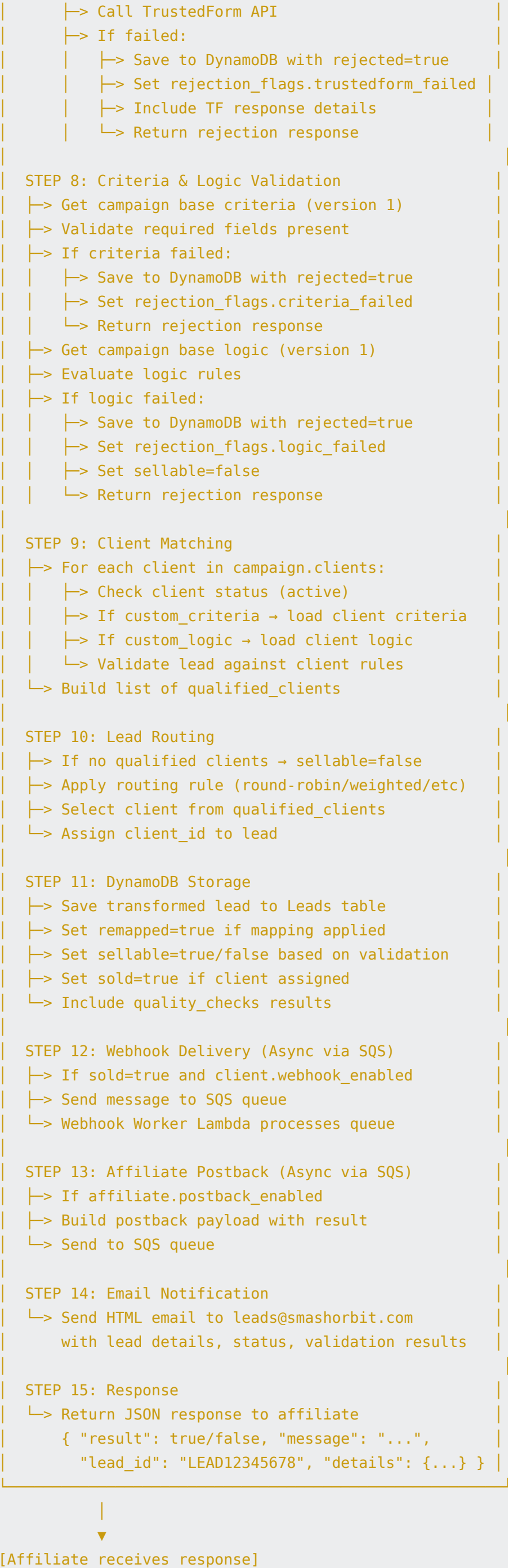
STEP 4: Affiliate Status & Endpoint Validation
└─> Get affiliate config from campaign.affiliates
└─> Check affiliate.status:
|   • If "test" + endpoint="/v2/lead/intake":
|   |   └─> Save to DynamoDB with rejected=true
|   |   └─> Set rejection_flags.test_affiliate_on_prod
|   |   └─> Skip validation (validation_skipped)
|   |   └─> Send notification
|   |   └─> Return 403 response
|   • If "live" + endpoint="/v2/lead/test":
|   |   └─> Save to DynamoDB with rejected=true
|   |   └─> Set rejection_flags.live_affiliate_on_test
|   |   └─> Skip validation (validation_skipped)
|   |   └─> Return 403 response
|   • If "inactive":
|   |   └─> Save to DynamoDB with rejected=true
|   |   └─> Set rejection_flags.inactive_affiliate
|   |   └─> Skip validation (validation_skipped)
|   |   └─> Send notification if prod endpoint
|   |   └─> Return 403 response
└─> Continue if validated

STEP 5: Cap Validation & Increment
└─> Check affiliate.cap_type (lifetime/daily/weekly/monthly)
└─> Check if current period expired → Reset counter
└─> If leads_sent_current_period >= cap_limit:
|   └─> Save to DynamoDB with rejected=true
|   └─> Set rejection_flags.cap_exceeded
|   └─> Skip validation (validation_skipped)
|   └─> Do NOT increment counter
|   └─> Send capacity warning notification
|   └─> Return 429 response
└─> Increment counter (atomic update) if not capped

STEP 6: Field Mapping
└─> Check if affiliate has custom_mapping_enabled
└─> If yes, apply field_mappings
|   (fname → first_name, contact_email → email)
└─> Create transformed_payload

STEP 7: Quality Checks (Plugins)
└─> Duplicate Check (if enabled)
|   └─> Query phone-index GSI
|   └─> Query email-index GSI
|   └─> If duplicate found:
|   |   └─> Save to DynamoDB with rejected=true
|   |   └─> Set rejection_flags.duplicate
|   |   └─> Skip IPQS/TF (validation_skipped)
|   |   └─> Return rejection response
└─> IPQS Validation (if enabled)
|   └─> Use credentials from environment variables
|   └─> Call IPQS API (phone, email, IP)
|   └─> Compare results with thresholds
|   └─> If failed:
|   |   └─> Save to DynamoDB with rejected=true
|   |   └─> Set rejection_flags.ipqs_failed
|   |   └─> Include IPQS response details
|   |   └─> Return rejection response
└─> TrustedForm Validation (if enabled)
|   └─> Use credentials from environment variables

```



Detailed Flow Steps

STEP 1: Raw Payload Logging

```

async function logRawPayload(payload, campaignId, affiliateId) {
  // Generate unique lead ID
  const leadId = generateLeadId(); // LEAD12345678
  const timestamp = new Date().toISOString();

  // Prepare Parquet record
  const record = {
    lead_id: leadId,
    timestamp: timestamp,
    campaign_id: campaignId,
    affiliate_id: affiliateId,
    raw_payload: payload,
    field_mappings: null, // Will be set after mapping
    transformed_payload: null, // Will be set after mapping
    remapped: false,
    tenant_id: process.env.TENANT_ID
  };

  // Append to daily Parquet file in S3
  const s3Key = `leads/${year}/${month}/${day}/leads-${year}-${month}-${day}.parquet`;
  await appendToParquet(s3Key, record);

  return leadId;
}

```

STEP 2-5: Campaign Key & Status Validation

```

async function validateCampaignKeyAndStatus(campaignKey, endpoint, rawPayload) {
  // STEP 2: Campaign Key Validation
  const campaign = await campaignsTable.query({
    IndexName: 'campaign-key-index', // GSI on campaign_key
    KeyConditionExpression: 'campaign_key = :key',
    ExpressionAttributeValues: { ':key': campaignKey }
  });

  if (!campaign || campaign.length === 0) {
    // Generate lead ID for tracking
    const leadId = generateLeadId();
    const timestamp = new Date().toISOString();

    // Save rejected lead to DynamoDB
    await saveRejectedLead({
      id: leadId,
      timestamp: timestamp,
      campaign_key: campaignKey,
      raw_payload: rawPayload,
      rejected: true,
      rejection_flags: {
        invalid_campaign_key: true
      },
      rejection_reason: 'Invalid campaign key provided',
      validation_skipped: true,
      endpoint: endpoint
    });

    // Send notification
    await sendNotification({
      type: 'invalid_campaign_key',
      campaign_key: campaignKey,
      lead_id: leadId,
      timestamp: timestamp
    });

    throw {
      statusCode: 401,
      error: 'Invalid campaign key',
      message: 'The provided campaign key does not exist or is malformed',
    };
  }
}

```

```
    lead_id: leadId // Return lead ID so it's trackable
  };
}

const campaignData = campaign[0];

// STEP 3: Campaign Status Check
if (campaignData.status !== 'active') {
  const leadId = generateLeadId();
  const timestamp = new Date().toISOString();

  // Save rejected lead to DynamoDB
  await saveRejectedLead({
    id: leadId,
    timestamp: timestamp,
    campaign_id: campaignData.id,
    campaign_key: campaignKey,
    raw_payload: rawPayload,
    rejected: true,
    rejection_flags: {
      inactive_campaign: true
    },
    rejection_reason: campaignData.rejection_messages?.inactive_campaign || 'Lead received after campaign was marked inactive',
    validation_skipped: true,
    endpoint: endpoint
  });

  // Send notification (especially in production)
  await sendNotification({
    type: 'inactive_campaign',
    campaign_id: campaignData.id,
    campaign_name: campaignData.name,
    lead_id: leadId,
    timestamp: timestamp,
    environment: process.env.ENVIRONMENT
  });

  throw {
    statusCode: 403,
    error: 'Campaign inactive',
    message: 'This campaign is no longer active',
    lead_id: leadId
  };
}

// Find affiliate by campaign_key in affiliates array
const affiliate = campaignData.affiliates.find(a => a.campaign_key === campaignKey);

if (!affiliate) {
  const leadId = generateLeadId();
  const timestamp = new Date().toISOString();

  await saveRejectedLead({
    id: leadId,
    timestamp: timestamp,
    campaign_id: campaignData.id,
    campaign_key: campaignKey,
    raw_payload: rawPayload,
    rejected: true,
    rejection_flags: {
      invalid_campaign_key: true
    },
    rejection_reason: 'Affiliate configuration not found for campaign key',
    validation_skipped: true
  });

  throw {
    statusCode: 401,
    error: 'Invalid campaign key',
    message: 'Affiliate configuration not found',
    lead_id: leadId
  };
}
```

```

}

// STEP 4: Affiliate Status & Endpoint Validation
const isProductionEndpoint = endpoint === '/v2/lead/intake';
const isTestEndpoint = endpoint === '/v2/lead/test';

if (affiliate.status === 'test' && isProductionEndpoint) {
  const leadId = generateLeadId();
  const timestamp = new Date().toISOString();

  // Save rejected lead to DynamoDB
  await saveRejectedLead({
    id: leadId,
    timestamp: timestamp,
    campaign_id: campaignData.id,
    affiliate_id: affiliate.affiliate_id,
    campaign_key: campaignKey,
    raw_payload: rawPayload,
    rejected: true,
    rejection_flags: {
      test_affiliate_on_prod: true
    },
    rejection_reason: campaignData.rejection_messages?.test_affiliate_on_prod || 'Test affiliate attempted to use production',
    validation_skipped: true,
    endpoint: endpoint
  });

  // Send notification to admin
  await sendNotification({
    type: 'unauthorized_access',
    affiliate_id: affiliate.affiliate_id,
    campaign_id: campaignData.id,
    status: 'test',
    endpoint: endpoint,
    lead_id: leadId,
    timestamp: timestamp
  });

  throw {
    statusCode: 403,
    error: 'Test mode active',
    message: 'Your affiliate account is still in test mode. Please use the test endpoint: POST /v2/lead/test',
    test_endpoint: process.env.API_BASE_URL + '/v2/lead/test',
    lead_id: leadId
  };
}

if (affiliate.status === 'live' && isTestEndpoint) {
  const leadId = generateLeadId();
  const timestamp = new Date().toISOString();

  // Save rejected lead to DynamoDB
  await saveRejectedLead({
    id: leadId,
    timestamp: timestamp,
    campaign_id: campaignData.id,
    affiliate_id: affiliate.affiliate_id,
    campaign_key: campaignKey,
    raw_payload: rawPayload,
    rejected: true,
    rejection_flags: {
      live_affiliate_on_test: true
    },
    rejection_reason: campaignData.rejection_messages?.live_affiliate_on_test || 'Live affiliate attempted to use test endpoint',
    validation_skipped: true,
    endpoint: endpoint
  });

  throw {
    statusCode: 403,
    error: 'Campaign is live',
    message: 'Your affiliate account is live. Please use the production endpoint: POST /v2/lead/intake',
  };
}

```

```

    production_endpoint: process.env.API_BASE_URL + '/v2/lead/intake',
    lead_id: leadId
  };
}

if (affiliate.status === 'inactive') {
  const leadId = generateLeadId();
  const timestamp = new Date().toISOString();

  // Save rejected lead to DynamoDB
  await saveRejectedLead({
    id: leadId,
    timestamp: timestamp,
    campaign_id: campaignData.id,
    affiliate_id: affiliate.affiliate_id,
    campaign_key: campaignKey,
    raw_payload: rawPayload,
    rejected: true,
    rejection_flags: {
      inactive_affiliate: true
    },
    rejection_reason: campaignData.rejection_messages?.inactive_affiliate || 'Lead received from inactive affiliate',
    validation_skipped: true,
    endpoint: endpoint
  });

  // Send notification if trying production endpoint
  if (isProductionEndpoint) {
    await sendNotification({
      type: 'unauthorized_access',
      affiliate_id: affiliate.affiliate_id,
      campaign_id: campaignData.id,
      status: 'inactive',
      endpoint: endpoint,
      lead_id: leadId,
      timestamp: timestamp
    });
  }

  throw {
    statusCode: 403,
    error: 'Affiliate inactive',
    message: 'Your affiliate access has been disabled for this campaign. Please contact support.',
    lead_id: leadId
  };
}

// STEP 5: Cap Validation (only for production endpoint)
if (isProductionEndpoint && affiliate.status === 'live' && affiliate.cap_limit) {
  const now = new Date();
  const currentDate = now.toISOString().split('T')[0]; // YYYY-MM-DD

  // Helper function to calculate period dates based on cap_type
  function calculatePeriod(capType, referenceDate = new Date()) {
    const period = { start: null, end: null };
    const ref = new Date(referenceDate);
    ref.setUTCHours(0, 0, 0, 0);

    switch (capType) {
      case 'daily':
        period.start = ref.toISOString().split('T')[0];
        ref.setUTCDate(ref.getUTCDate() + 1);
        period.end = ref.toISOString().split('T')[0];
        break;

      case 'weekly':
        // Start of week (Monday)
        const dayOfWeek = ref.getUTCDay();
        const diff = dayOfWeek === 0 ? -6 : 1 - dayOfWeek; // Adjust to Monday
        ref.setUTCDate(ref.getUTCDate() + diff);
        period.start = ref.toISOString().split('T')[0];
        ref.setUTCDate(ref.getUTCDate() + 7);
    }
  }
}

```

```

        period.end = ref.toISOString().split('T')[0];
        break;

    case 'monthly':
        // Start of month
        ref.setUTCDate(1);
        period.start = ref.toISOString().split('T')[0];
        ref.setUTCMonth(ref.getUTCMonth() + 1);
        period.end = ref.toISOString().split('T')[0];
        break;

    case 'overall':
        // No reset - lifetime cap
        period.start = null;
        period.end = null;
        break;
    }

    return period;
}

// Check if current period has expired and needs reset
let needsReset = false;

if (affiliate.cap_type === 'overall') {
    // Overall cap never resets, use total_leads_sent
    needsReset = false;
} else if (!affiliate.cap_period_end || currentDate >= affiliate.cap_period_end) {
    // Period expired, calculate new period
    needsReset = true;
}

if (needsReset) {
    const newPeriod = calculatePeriod(affiliate.cap_type, now);

    // Reset counter for new period (atomic update)
    await campaignsTable.update({
        Key: { id: campaignData.id },
        UpdateExpression: 'SET affiliates[#idx].leads_sent_current_period = :zero, affiliates[#idx].cap_period_start = :start, ExpressionAttributeNames: { '#idx': campaignData.affiliates.indexOf(affiliate) }, ExpressionAttributeValues: { ':zero': 0, ':start': newPeriod.start, ':end': newPeriod.end }
    });

    affiliate.leads_sent_current_period = 0;
    affiliate.cap_period_start = newPeriod.start;
    affiliate.cap_period_end = newPeriod.end;
}

// Determine current count based on cap type
const currentCount = affiliate.cap_type === 'overall'
    ? affiliate.total_leads_sent
    : affiliate.leads_sent_current_period;

// Check if cap is reached
if (currentCount >= affiliate.cap_limit) {
    const leadId = generateLeadId();
    const timestamp = new Date().toISOString();

    // Save rejected lead to DynamoDB (DO NOT increment counter)
    await saveRejectedLead({
        id: leadId,
        timestamp: timestamp,
        campaign_id: campaignData.id,
        affiliate_id: affiliate.affiliate_id,
        campaign_key: campaignKey,
        raw_payload: rawPayload,
    });
}

```

```

    rejected: true,
    rejection_flags: {
      cap_exceeded: true
    },
    rejection_reason: campaignData.rejection_messages?.cap_exceeded || `${affiliate.cap_type} lead cap exceeded`,
    rejection_details: {
      cap_type: affiliate.cap_type,
      cap_limit: affiliate.cap_limit,
      current_count: currentCount,
      period_start: affiliate.cap_period_start,
      period_end: affiliate.cap_period_end
    },
    validation_skipped: true,
    endpoint: endpoint
  });

  // Send capacity warning notification
  await sendNotification({
    type: 'capacity_reached',
    affiliate_id: affiliate.affiliate_id,
    campaign_id: campaignData.id,
    cap_type: affiliate.cap_type,
    cap_limit: affiliate.cap_limit,
    current_count: currentCount,
    lead_id: leadId,
    timestamp: timestamp
  });

  // Calculate reset time (null for overall cap)
  let resetTime = null;
  if (affiliate.cap_type !== 'overall' && affiliate.cap_period_end) {
    const reset = new Date(affiliate.cap_period_end);
    reset.setUTCHours(0, 0, 0, 0);
    resetTime = reset.toISOString();
  }

  throw {
    statusCode: 429,
    error: `${affiliate.cap_type.charAt(0).toUpperCase() + affiliate.cap_type.slice(1)} cap reached`,
    message: `You have reached your ${affiliate.cap_type} lead cap for this campaign`,
    lead_id: leadId,
    details: {
      cap_type: affiliate.cap_type,
      cap_limit: affiliate.cap_limit,
      leads_sent: currentCount,
      remaining: 0,
      reset_time: resetTime,
      period_start: affiliate.cap_period_start,
      period_end: affiliate.cap_period_end
    }
  };
}

// Cap NOT exceeded - Increment counters (atomic update)
const updateExpr = affiliate.cap_type === 'lifetime'
  ? 'ADD affiliates[#idx].total_leads_sent :inc'
  : 'ADD affiliates[#idx].leads_sent_current_period :inc, affiliates[#idx].total_leads_sent :inc';

await campaignsTable.update({
  Key: { id: campaignData.id },
  UpdateExpression: updateExpr,
  ExpressionAttributeNames: {
    '#idx': campaignData.affiliates.indexOf(affiliate)
  },
  ExpressionAttributeValues: {
    ':inc': 1
  }
});

// Send capacity warnings at 80% and 90%
const newCount = currentCount + 1;
const percentage = (newCount / affiliate.cap_limit) * 100;

```

```

if (percentage >= 80 && currentCount < affiliate.cap_limit * 0.8) {
  await sendNotification({
    type: 'capacity_warning',
    affiliate_id: affiliate.affiliate_id,
    campaign_id: campaignData.id,
    cap_type: affiliate.cap_type,
    percentage: 80,
    current_count: newCount,
    cap_limit: affiliate.cap_limit
  });
} else if (percentage >= 90 && currentCount < affiliate.cap_limit * 0.9) {
  await sendNotification({
    type: 'capacity_warning',
    affiliate_id: affiliate.affiliate_id,
    campaign_id: campaignData.id,
    cap_type: affiliate.cap_type,
    percentage: 90,
    current_count: newCount,
    cap_limit: affiliate.cap_limit
  });
}
}

return { campaign: campaignData, affiliate };
}

// Helper function to save rejected leads
async function saveRejectedLead(leadData) {
  // Save to DynamoDB Leads table
  await leadsTable.put({
    ...leadData,
    sellable: false,
    sold: false,
    created_at: leadData.timestamp,
    // Include all rejection tracking fields
    rejected: true,
    rejection_flags: leadData.rejection_flags || {},
    rejection_reason: leadData.rejection_reason,
    rejection_details: leadData.rejection_details || {},
    validation_skipped: leadData.validation_skipped || false
  });

  // Also log to S3 for audit trail
  await logToS3({
    type: 'rejected_lead',
    ...leadData
  });
}

if (isProductionEndpoint && affiliate.status === 'live' && affiliate.cap_limit) {
  const now = new Date();
  const currentDate = now.toISOString().split('T')[0]; // YYYY-MM-DD

  // Helper function to calculate period dates based on cap_type
  function calculatePeriod(capType, referenceDate = new Date()) {
    const period = { start: null, end: null };
    const ref = new Date(referenceDate);
    ref.setUTCHours(0, 0, 0, 0);

    switch (capType) {
      case 'daily':
        period.start = ref.toISOString().split('T')[0];
        ref.setUTCDate(ref.getUTCDate() + 1);
        period.end = ref.toISOString().split('T')[0];
        break;

      case 'weekly':
        // Start of week (Monday)
        const dayOfWeek = ref.getUTCDay();
        const diff = dayOfWeek === 0 ? -6 : 1 - dayOfWeek; // Adjust to Monday
        ref.setUTCDate(ref.getUTCDate() + diff);
        period.start = ref.toISOString().split('T')[0];

```

```

        ref.setUTCDate(ref.getUTCDate() + 7);
        period.end = ref.toISOString().split('T')[0];
        break;

    case 'monthly':
        // Start of month
        ref.setUTCDate(1);
        period.start = ref.toISOString().split('T')[0];
        ref.setUTCMonth(ref.getUTCMonth() + 1);
        period.end = ref.toISOString().split('T')[0];
        break;

    case 'overall':
        // No reset - lifetime cap
        period.start = null;
        period.end = null;
        break;
    }

    return period;
}

// Check if current period has expired and needs reset
let needsReset = false;

if (affiliate.cap_type === 'overall') {
    // Overall cap never resets, use total_leads_sent
    needsReset = false;
} else if (!affiliate.cap_period_end || currentDate >= affiliate.cap_period_end) {
    // Period expired, calculate new period
    needsReset = true;
}

if (needsReset) {
    const newPeriod = calculatePeriod(affiliate.cap_type, now);

    // Reset counter for new period (atomic update)
    await campaignsTable.update({
        Key: { id: campaignData.id },
        UpdateExpression: 'SET affiliates[#idx].leads_sent_current_period = :zero, affiliates[#idx].cap_period_start = :start,
        ExpressionAttributeNames: {
            '#idx': campaignData.affiliates.indexOf(affiliate)
        },
        ExpressionAttributeValues: {
            ':zero': 0,
            ':start': newPeriod.start,
            ':end': newPeriod.end
        }
    });

    affiliate.leads_sent_current_period = 0;
    affiliate.cap_period_start = newPeriod.start;
    affiliate.cap_period_end = newPeriod.end;
}

// Determine current count based on cap type
const currentCount = affiliate.cap_type === 'overall'
    ? affiliate.total_leads_sent
    : affiliate.leads_sent_current_period;

// Check if cap is reached
if (currentCount >= affiliate.cap_limit) {
    // Send capacity warning notification
    await sendNotification({
        type: 'capacity_reached',
        affiliate_id: affiliate.affiliate_id,
        campaign_id: campaignData.id,
        cap_type: affiliate.cap_type,
        cap_limit: affiliate.cap_limit,
        current_count: currentCount,
        timestamp: new Date().toISOString()
    });
}

```

```

// Calculate reset time (null for overall cap)
let resetTime = null;
if (affiliate.cap_type !== 'overall' && affiliate.cap_period_end) {
  const reset = new Date(affiliate.cap_period_end);
  reset.setUTCHours(0, 0, 0, 0);
  resetTime = reset.toISOString();
}

throw {
  statusCode: 429,
  error: `${affiliate.cap_type.charAt(0).toUpperCase() + affiliate.cap_type.slice(1)} cap reached`,
  message: `You have reached your ${affiliate.cap_type} lead cap for this campaign`,
  details: {
    cap_type: affiliate.cap_type,
    cap_limit: affiliate.cap_limit,
    leads_sent: currentCount,
    remaining: 0,
    reset_time: resetTime,
    period_start: affiliate.cap_period_start,
    period_end: affiliate.cap_period_end
  }
};
}

// Increment counters (atomic update)
const updateExpr = affiliate.cap_type === 'overall'
  ? 'ADD affiliates[#idx].total_leads_sent :inc'
  : 'ADD affiliates[#idx].leads_sent_current_period :inc, affiliates[#idx].total_leads_sent :inc';

await campaignsTable.update({
  Key: { id: campaignData.id },
  UpdateExpression: updateExpr,
  ExpressionAttributeNames: {
    '#idx': campaignData.affiliates.indexOf(affiliate)
  },
  ExpressionAttributeValues: {
    ':inc': 1
  }
});

// Send capacity warnings at 80% and 90%
const newCount = currentCount + 1;
const percentage = (newCount / affiliate.cap_limit) * 100;

if (percentage >= 80 && currentCount < affiliate.cap_limit * 0.8) {
  await sendNotification({
    type: 'capacity_warning',
    affiliate_id: affiliate.affiliate_id,
    campaign_id: campaignData.id,
    cap_type: affiliate.cap_type,
    percentage: 80,
    current_count: newCount,
    cap_limit: affiliate.cap_limit
  });
} else if (percentage >= 90 && currentCount < affiliate.cap_limit * 0.9) {
  await sendNotification({
    type: 'capacity_warning',
    affiliate_id: affiliate.affiliate_id,
    campaign_id: campaignData.id,
    cap_type: affiliate.cap_type,
    percentage: 90,
    current_count: newCount,
    cap_limit: affiliate.cap_limit
  });
}
}

return { campaign: campaignData, affiliate };
}

```

STEP 6: Field Mapping

```
async function applyFieldMapping(payload, affiliate) {
  let transformedPayload = { ...payload };
  let remapped = false;
  let fieldMappings = {};
  let valueMappings = {};

  if (affiliate.custom_mapping_enabled) {
    // STEP 6a: Apply field-level mappings (key transformations)
    if (affiliate.field_mappings) {
      for (const [affiliateField, campaignField] of Object.entries(affiliate.field_mappings)) {
        if (payload[affiliateField] !== undefined) {
          transformedPayload[campaignField] = payload[affiliateField];
          fieldMappings[affiliateField] = campaignField;

          // Remove old field name if different
          if (affiliateField !== campaignField) {
            delete transformedPayload[affiliateField];
          }

          remapped = true;
        }
      }
    }

    // STEP 6b: Apply value-level transformations
    if (affiliate.value_mappings) {
      for (const [fieldName, valueMap] of Object.entries(affiliate.value_mappings)) {
        const currentValue = transformedPayload[fieldName];

        if (currentValue !== undefined && valueMap[currentValue]) {
          // Transform the value to canonical form
          const canonicalValue = valueMap[currentValue];
          transformedPayload[fieldName] = canonicalValue;

          valueMappings[fieldName] = {
            original: currentValue,
            transformed: canonicalValue
          };

          remapped = true;
        }
      }
    }
  }

  // Update S3 Parquet record with mappings
  await updateParquetRecord(leadId, {
    field_mappings: remapped ? fieldMappings : null,
    transformed_payload: transformedPayload,
    remapped: remapped
  });

  return { transformedPayload, remapped };
}
```

STEP 4: Quality Checks (Plugins)

```
async function executeQualityChecks(lead, campaign, tenantId) {
  const results = {
    ipqs: { enabled: false, passed: true, results: {} },
    trustedform: { enabled: false, passed: true, results: {} },
    duplicate: { enabled: false, passed: true, duplicate_found: false }
  };

  // Duplicate Check
  if (campaign.plugins.duplicate_check?.enabled) {
```

```
results.duplicate.enabled = true;
const config = campaign.plugins.duplicate_check.config;

let duplicateFound = false;

if (config.phone_enabled) {
  const phoneDups = await leadsTable.queryByGsi('phone-index', 'phone', lead.phone);
  if (phoneDups.length > 0) {
    duplicateFound = true;
  }
}

if (config.email_enabled) {
  const emailDups = await leadsTable.queryByGsi('email-index', 'email', lead.email);
  if (emailDups.length > 0) {
    if (config.operator === 'OR') {
      duplicateFound = true;
    } else if (config.operator === 'AND' && duplicateFound) {
      duplicateFound = true; // Both phone and email match
    } else {
      duplicateFound = false; // Only email matches, not phone
    }
  }
}

results.duplicate.duplicate_found = duplicateFound;
results.duplicate.passed = !duplicateFound;

if (duplicateFound && config.action === 'reject') {
  throw new ValidationError('Duplicate lead detected', results);
}
}

// IPQS Check
if (campaign.plugins.ipqs?.enabled) {
  results.ipqs.enabled = true;
  const config = campaign.plugins.ipqs.config;

  // Credentials are already loaded in Lambda environment variables during deployment
  const ipqsResults = await ipqsService.validate(
    lead.ip_address,
    lead.phone,
    lead.email,
    {
      apiKey: process.env.IPQS_API_KEY,
      baseUrl: process.env.IPQS_BASE_URL
    }
  );

  results.ipqs.results = ipqsResults;

  // Check thresholds
  const passed = validateThresholds(ipqsResults, config.thresholds);
  results.ipqs.passed = passed;

  if (!passed) {
    throw new ValidationError('IPQS validation failed', results);
  }
}

// TrustedForm Check
if (campaign.plugins.trustedform?.enabled) {
  results.trustedform.enabled = true;

  // Credentials are already loaded in Lambda environment variables during deployment
  const tfResults = await trustedformService.validate(
    lead.trusted_form_cert_id,
    lead.phone,
    {
      username: process.env.TRUSTEDFORM_USERNAME,
      password: process.env.TRUSTEDFORM_PASSWORD
    }
  );
}
```

```

    );

    results.trustedform.results = tfResults;
    results.trustedform.passed = tfResults.outcome === 'success';

    if (!results.trustedform.passed) {
        throw new ValidationError('TrustedForm validation failed', results);
    }
}

return results;
}

```

STEP 5: Criteria & Logic Validation

```

async function validateCriteriaAndLogic(lead, campaign) {
    // Get base criteria (version 1)
    const criteriaConfig = await configCatalog.get({
        campaign_id: campaign.id,
        config_type: `criteria#v${campaign.current_criteria_version}`
    });

    // Validate required fields
    for (const question of criteriaConfig.config.questions) {
        if (question.required && !lead[question.key]) {
            throw new ValidationError(`Missing required field: ${question.display_name}`);
        }
    }

    // Get base logic (version 1)
    const logicConfig = await configCatalog.get({
        campaign_id: campaign.id,
        config_type: `logic#v${campaign.current_logic_version}`
    });

    // Evaluate logic rules
    const logicPassed = evaluateLogicRules(lead, logicConfig.config);

    return {
        criteria_passed: true,
        criteria_version: campaign.current_criteria_version,
        logic_passed: logicPassed,
        logic_version: campaign.current_logic_version
    };
}

function evaluateLogicRules(lead, logicConfig) {
    const ruleResults = [];

    for (const rule of logicConfig.rules) {
        let ruleMatches = false;

        for (const group of rule.groups) {
            const conditionResults = group.conditions.map(condition =>
                evaluateCondition(lead[condition.field], condition.operator, condition.value)
            );

            const groupMatches = group.operator === 'AND'
                ? conditionResults.every(r => r)
                : conditionResults.some(r => r);

            if (groupMatches) {
                ruleMatches = true;
                break; // OR between groups
            }
        }

        ruleResults.push(ruleMatches);
    }
}

```

```

// Combine rule results
return logicConfig.combine_rules === 'AND'
  ? ruleResults.every(r => r)
  : ruleResults.some(r => r);
}

```

STEP 6: Client Matching

```

async function matchClients(lead, campaign) {
  const qualifiedClients = [];

  for (const clientConfig of campaign.clients) {
    if (clientConfig.status !== 'active') {
      continue; // Skip inactive clients
    }

    let clientQualifies = true;

    // Check custom criteria
    if (clientConfig.custom_criteria_enabled) {
      const clientCriteria = await configCatalog.get({
        campaign_id: campaign.id,
        config_type: `criteria#v${clientConfig.criteria_version}`
      });

      // Validate required fields for this client
      for (const question of clientCriteria.config.questions) {
        if (question.required && !lead[question.key]) {
          clientQualifies = false;
          break;
        }
      }
    }

    // Check custom logic
    if (clientQualifies && clientConfig.custom_logic_enabled) {
      const clientLogic = await configCatalog.get({
        campaign_id: campaign.id,
        config_type: `logic#v${clientConfig.logic_version}`
      });

      const logicPassed = evaluateLogicRules(lead, clientLogic.config);
      if (!logicPassed) {
        clientQualifies = false;
      }
    }

    if (clientQualifies) {
      qualifiedClients.push(clientConfig);
    }
  }

  return qualifiedClients;
}

```

STEP 7: Lead Routing

```

async function routeLead(qualifiedClients, campaign) {
  if (qualifiedClients.length === 0) {
    return null; // No client assigned, sellable = false
  }

  let selectedClient = null;

  switch (campaign.routing_type) {

```

```

case 'round_robin':
  // Round-robin: rotate through clients
  const index = campaign.routing_state.last_client_index || 0;
  selectedClient = qualifiedClients[index % qualifiedClients.length];

  // Update routing state
  campaign.routing_state.last_client_index = (index + 1) % qualifiedClients.length;
  await campaignsTable.update(campaign);
  break;

case 'weighted':
  // Weighted: distribute based on percentages/ratios
  selectedClient = selectWeightedClient(qualifiedClients);
  break;

case 'priority':
  // Priority: highest priority (lowest number) first
  selectedClient = qualifiedClients.sort((a, b) =>
    a.routing_priority - b.routing_priority
  )[0];
  break;

case 'traffic_based':
  // Traffic-based: custom logic based on current traffic
  selectedClient = selectTrafficBasedClient(qualifiedClients, campaign);
  break;

default:
  selectedClient = qualifiedClients[0];
}

// Update client count
if (!campaign.routing_state.client_counts) {
  campaign.routing_state.client_counts = {};
}
campaign.routing_state.client_counts[selectedClient.client_id] =
  (campaign.routing_state.client_counts[selectedClient.client_id] || 0) + 1;

return selectedClient;
}

function selectWeightedClient(qualifiedClients) {
  // Calculate total weight
  const totalWeight = qualifiedClients.reduce((sum, c) => sum + c.routing_weight, 0);

  // Generate random number
  let random = Math.random() * totalWeight;

  // Select client based on weight
  for (const client of qualifiedClients) {
    random -= client.routing_weight;
    if (random <= 0) {
      return client;
    }
  }

  return qualifiedClients[0]; // Fallback
}

```

STEP 8: DynamoDB Storage

```

async function storeLead(leadData) {
  const leadRecord = {
    id: leadData.lead_id,
    timestamp: leadData.timestamp,
    campaign_id: leadData.campaign_id,
    affiliate_id: leadData.affiliate_id,
    client_id: leadData.client_id || null,
    created_at: leadData.timestamp,
  }
}

```

```

date: leadData.timestamp.split('T')[0],
time: leadData.timestamp.split('T')[1].split('.')[0],

// Transformed lead data
...leadData.transformed_payload,

// S3 reference
s3_raw_payload_path: `s3://bucket/leads/${year}/${month}/${day}/leads-${year}-${month}-${day}.parquet`,
remapped: leadData.remapped,

// Quality check results
quality_checks: leadData.quality_checks,

// Criteria/logic validation
criteria_passed: leadData.criteria_passed,
criteria_version: leadData.criteria_version,
logic_passed: leadData.logic_passed,
logic_version: leadData.logic_version,

// Routing
routing_rule: leadData.routing_rule,
sellable: leadData.sellable,
sold: leadData.sold,
sold_at: leadData.sold_at,

// Webhooks (updated later)
webhook_sent: false,
postback_sent: false,
email_sent: false,

// Metadata
test_mode: leadData.test_mode || false,
tenant_id: leadData.tenant_id
};

await leadsTable.put(leadRecord);
return leadRecord;
}

```

STEP 9 & 10: Async Webhooks (SQS)

```

async function queueWebhooks(lead, client, affiliate) {
  const webhookMessages = [];

  // Client webhook
  if (lead.sold && client.webhook_enabled) {
    webhookMessages.push({
      type: 'client_webhook',
      lead_id: lead.id,
      client_id: client.client_id,
      webhook_url: client.webhook_url,
      retry_attempts: client.webhook_retry_attempts || 3,
      payload: buildClientWebhookPayload(lead)
    });
  }

  // Affiliate postback
  if (affiliate.postback_enabled) {
    webhookMessages.push({
      type: 'affiliate_postback',
      lead_id: lead.id,
      affiliate_id: affiliate.affiliate_id,
      postback_url: affiliate.postback_url,
      payload: buildAffiliatePostbackPayload(lead)
    });
  }

  // Send to SQS
  for (const message of webhookMessages) {

```

```

    await sqsClient.sendMessage({
      QueueUrl: process.env.WEBHOOK_QUEUE_URL,
      MessageBody: JSON.stringify(message)
    });
  }
}

function buildClientWebhookPayload(lead) {
  return {
    lead_id: lead.id,
    timestamp: lead.timestamp,
    first_name: lead.first_name,
    last_name: lead.last_name,
    email: lead.email,
    phone: lead.phone,
    state: lead.state,
    ip_address: lead.ip_address,
    rideshare_company: lead.rideshare_company,
    // ... all relevant lead data
    quality_checks: lead.quality_checks,
    sellable: lead.sellable
  };
}

function buildAffiliatePostbackPayload(lead) {
  const status = lead.sold ? 'accepted' : 'rejected';
  let rejectionReason = null;

  if (!lead.sold) {
    if (!lead.quality_checks.duplicate.passed) {
      rejectionReason = 'Duplicate lead detected';
    } else if (!lead.quality_checks.ipqs.passed) {
      rejectionReason = 'IPQS validation failed';
    } else if (!lead.quality_checks.trustedform.passed) {
      rejectionReason = 'TrustedForm validation failed';
    } else if (!lead.logic_passed) {
      rejectionReason = 'Logic validation failed';
    } else {
      rejectionReason = 'No matching client';
    }
  }

  return {
    lead_id: lead.id,
    status: status,
    rejection_reason: rejectionReason,
    timestamp: lead.timestamp
  };
}

```

STEP 11: Email Notification

```

async function sendEmailNotification(lead, campaign, client) {
  const emailHtml = buildEmailTemplate(lead, campaign, client);

  await sesClient.sendEmail({
    Source: 'noreply@smashorbit.com',
    Destination: {
      ToAddresses: [campaign.notification_emails?.leads || 'leads@smashorbit.com']
    },
    Message: {
      Subject: {
        Data: `Lead ${lead.sold ? 'SOLD' : 'REJECTED'} - ${lead.id}`
      },
      Body: {
        Html: {
          Data: emailHtml
        }
      }
    }
  });
}

```

```
    }
  });

  // Update lead record
  await leadsTable.update({
    id: lead.id,
    timestamp: lead.timestamp
  }, {
    email_sent: true
  });
}

function buildEmailTemplate(lead, campaign, client) {
  return `
<!DOCTYPE html>
<html>
<head>
  <style>
    body { font-family: Arial, sans-serif; }
    .status-sold { color: green; font-weight: bold; }
    .status-rejected { color: red; font-weight: bold; }
    table { border-collapse: collapse; width: 100%; }
    th, td { border: 1px solid #ddd; padding: 8px; text-align: left; }
    th { background-color: #4CAF50; color: white; }
  </style>
</head>
<body>
  <h2>Lead ${lead.sold ? 'SOLD' : 'REJECTED'}</h2>

  <h3>Lead Information</h3>
  <table>
    <tr><th>Field</th><th>Value</th></tr>
    <tr><td>Lead ID</td><td>${lead.id}</td></tr>
    <tr><td>Campaign</td><td>${campaign.name}</td></tr>
    <tr><td>Status</td><td class="${lead.sold ? 'status-sold' : 'status-rejected'}">
      ${lead.sold ? 'SOLD' : 'REJECTED'}
    </td></tr>
    <tr><td>Client</td><td>${client.name}</td></tr>` : ''}
    <tr><td>Name</td><td>${lead.first_name} ${lead.last_name}</td></tr>
    <tr><td>Email</td><td>${lead.email}</td></tr>
    <tr><td>Phone</td><td>${lead.phone}</td></tr>
    <tr><td>State</td><td>${lead.state}</td></tr>
  </table>

  <h3>Quality Checks</h3>
  <table>
    <tr>
      <td>IPQS</td>
      <td>${lead.quality_checks.ipqs.passed ? '✅ PASSED' : '❌ FAILED'}</td>
    </tr>
    <tr>
      <td>TrustedForm</td>
      <td>${lead.quality_checks.trustedform.passed ? '✅ PASSED' : '❌ FAILED'}</td>
    </tr>
    <tr>
      <td>Duplicate Check</td>
      <td>${lead.quality_checks.duplicate.passed ? '✅ PASSED' : '❌ FAILED'}</td>
    </tr>
  </table>

  <h3>Criteria Validation</h3>
  <table>
    <tr>
      <td>Criteria</td>
      <td>${lead.criteria_passed ? '✅ PASSED' : '❌ FAILED'}</td>
    </tr>
    <tr>
      <td>Logic</td>
      <td>${lead.logic_passed ? '✅ PASSED' : '❌ FAILED'}</td>
    </tr>
  </table>
</body>
```

```
</html>
`
}
```

API Design

Base URL

Production: <https://api.smashorbit.com/v2>
Development: <https://api-dev.smashorbit.com/v2>

Authentication

- API Key in header: `X-API-Key: {affiliate_api_key}`
- Or Campaign ID + Affiliate ID in payload

Endpoints

1. Lead Intake (Production)

`POST /v2/lead/intake`

Headers:

`Content-Type: application/json`

Request Body:

```
{
  "campaign_key": "CAMPKEY_ABC123XYZ789", // Required: Unique API key provided when affiliate linked to campaign

  // Lead data (using affiliate's field names if custom mapping enabled)
  "fname": "John",
  "lname": "Doe",
  "contact_email": "john@example.com",
  "phone_num": "1234567890",
  "state_value": "TX",
  "ip": "1.2.3.4",
  "trusted_form_cert": "abc123def456",

  // Campaign-specific fields
  "rideshare_abuse": true,
  "rideshare_company_value": "uber",
  "has_attorney": false,

  // Marketing data
  "utm_source": "google",
  "utm_campaign": "rideshare-q1"
}
```

Success Response (200):

```
{
  "result": true,
  "message": "lead accepted",
  "lead_id": "LEAD12345678",
  "timestamp": "2026-02-04T12:30:45.123Z",
  "sold": true,
  "client": "Smith & Associates Law Firm"
}
```

Rejection Response (200):

```
{
  "result": false,
  "message": "lead rejected",
  "lead_id": "LEAD12345678",
  "timestamp": "2026-02-04T12:30:45.123Z",
  "rejection_reason": "IPQS validation failed",
  "details": {
    "ipqs": {
      "phone": "VOIP number, Risky number",
      "ip": "High fraud score, Proxy detected"
    }
  }
}
```

Error Responses:

```
// 401 Unauthorized - Invalid campaign key
{
  "error": "Invalid campaign key",
  "message": "The provided campaign key does not exist or is malformed"
}

// 403 Forbidden - Campaign inactive
{
  "error": "Campaign inactive",
  "message": "This campaign is no longer active"
}

// 403 Forbidden - Affiliate in test mode
{
  "error": "Test mode active",
  "message": "Your affiliate account is still in test mode. Please use the test endpoint: POST /v2/lead/test",
  "test_endpoint": "https://api.yourdomain.com/v2/lead/test"
}

// 403 Forbidden - Affiliate is live (trying to use test endpoint)
{
  "error": "Campaign is live",
  "message": "Your affiliate account is live. Please use the production endpoint: POST /v2/lead/intake",
  "production_endpoint": "https://api.yourdomain.com/v2/lead/intake"
}

// 403 Forbidden - Affiliate inactive
{
  "error": "Affiliate inactive",
  "message": "Your affiliate access has been disabled for this campaign. Please contact support."
}

// 429 Too Many Requests - Cap reached
{
  "error": "Weekly cap reached", // Varies by cap_type: Daily/Weekly/Monthly/Lifetime
  "message": "You have reached your weekly lead cap for this campaign",
  "details": {
    "cap_type": "weekly",
    "cap_limit": 500,
    "leads_sent": 500,
    "remaining": 0,
    "reset_time": "2026-02-10T00:00:00.000Z", // null for 'lifetime' cap type
    "period_start": "2026-02-03",
    "period_end": "2026-02-10"
  }
}

// 400 Bad Request - Missing required fields
{
  "error": "Missing required field: email"
```

```
}

// 429 Too Many Requests - Daily cap reached
{
  "error": "Affiliate daily cap reached (100 leads per day)"
}

// 500 Internal Server Error
{
  "error": "Internal server error processing lead"
}
```

2. Lead Intake (Test Mode)

POST /v2/lead/test

Purpose: Test lead intake with configurable validation bypass. Accepts leads from affiliates in any status (test, live, or inactive) to allow troubleshooting.

Headers:

Content-Type: application/json

Request Body: Same as production endpoint (must include campaign_key)

```
{
  "campaign_key": "CAMPKEY_ABC123XYZ789",
  "fname": "Jane",
  "lname": "Test",
  // ... other fields
}
```

Behavior: - Accepts requests from affiliates in **any status** (test, live, inactive) - Uses campaign's `test_config` settings - Can bypass quality checks, criteria, duplicate detection - Can skip webhooks and email notifications - Still logs to S3 and DynamoDB with `test_mode: true` flag - Does **not** increment affiliate's `leads_sent_today` counter

Response: Same format as production endpoint

Management APIs

All management endpoints require authentication (API key or JWT token for admin users).

Campaign Management

Campaign Creation Workflow:

- 1. **Browse Templates** - GET `/v2/templates` to view available criteria and logic templates
- 2. **Select or Create** - Choose a template or create custom configuration
- 3. **Create Campaign** - POST `/v2/campaigns` with criteria and logic source (template or custom)
- 4. **System Creates** - Version 1 of both criteria and logic configs are created in Config Catalog
- 5. **Campaign Active** - Campaign now has `current_criteria_version: 1` and `current_logic_version: 1`

Note: Campaigns **must** have both criteria and logic configured at creation. You cannot create a campaign without specifying how leads will be validated.

Create Campaign

POST /v2/campaigns

Request:

```
{
  "name": "Rideshare Abuse Campaign Q1 2026",
```

```
"status": "active",
"routing_type": "round_robin",

// Criteria Configuration (choose one approach)
"criteria": {
  "source": "template",          // "template" | "custom"
  "template_id": "TPL12345678"  // Required if source=template
  // OR for custom:
  // "source": "custom",
  // "config": { "questions": [...] }
},

// Logic Configuration (choose one approach)
"logic": {
  "source": "template",
  "template_id": "TPL87654321"
  // OR for custom:
  // "source": "custom",
  // "config": { "rules": [...] }
}
}
```

Response (201):

```
{
  "success": true,
  "campaign": {
    "id": "CAMP12345678",
    "name": "Rideshare Abuse Campaign Q1 2026",
    "status": "active",
    "routing_type": "round_robin",
    "current_criteria_version": 1,    // Created from template or custom config
    "current_logic_version": 1,      // Created from template or custom config
    "created_at": "2026-02-05T10:00:00.000Z"
  },
  "criteria_config": {
    "version": 1,
    "source": "template",
    "template_id": "TPL12345678",
    "template_name": "Standard Rideshare Intake"
  },
  "logic_config": {
    "version": 1,
    "source": "template",
    "template_id": "TPL87654321",
    "template_name": "Rideshare Qualification Logic"
  }
}
```

Update Campaign

PUT /v2/campaigns/{campaign_id}

Request:

```
{
  "name": "Updated Campaign Name",
  "status": "inactive",
  "routing_type": "weighted"
}
```

Response (200):

```
{
  "success": true,
  "campaign": { /* updated campaign object */ }
}
```

Get Campaign

```
GET /v2/campaigns/{campaign_id}
```

Response (200):

```
{
  "success": true,
  "campaign": {
    "id": "CAMP12345678",
    "name": "Rideshare Abuse Campaign Q1 2026",
    "status": "active",
    "affiliates": [ /* array of affiliate configs */ ],
    "clients": [ /* array of client configs */ ],
    "plugins": { /* plugin configurations */ },
    "routing_type": "round_robin",
    "current_criteria_version": 1,
    "current_logic_version": 1
  }
}
```

List Campaigns

```
GET /v2/campaigns?status=active&limit=50&offset=0
```

Response (200):

```
{
  "success": true,
  "campaigns": [ /* array of campaign objects */ ],
  "total": 15,
  "limit": 50,
  "offset": 0
}
```

Delete Campaign

```
DELETE /v2/campaigns/{campaign_id}
```

Response (200):

```
{
  "success": true,
  "message": "Campaign deleted successfully"
}
```

Affiliate Management

Create Affiliate

POST /v2/affiliates

Request:

```
{
  "name": "Partner ABC Marketing",
  "code": "ABC123",
  "status": "active",
  "contact_email": "partner@abc.com",
  "contact_phone": "5551234567"
}
```

Response (201):

```
{
  "success": true,
  "affiliate": {
    "id": "AFF12345678",
    "name": "Partner ABC Marketing",
    "code": "ABC123",
    "status": "active",
    "created_at": "2026-02-05T10:00:00.000Z"
  }
}
```

Update Affiliate

PUT /v2/affiliates/{affiliate_id}

Request:

```
{
  "name": "Updated Affiliate Name",
  "status": "inactive"
}
```

Response (200):

```
{
  "success": true,
  "affiliate": { /* updated affiliate object */ }
}
```

Get Affiliate

GET /v2/affiliates/{affiliate_id}

Response (200):

```
{
  "success": true,
  "affiliate": {
    "id": "AFF12345678",
    "name": "Partner ABC Marketing",
    "code": "ABC123",
    "status": "active",

```

```
    "campaigns": ["CAMP12345678"],
    "total_leads_sent": 1250,
    "acceptance_rate": 78.4
  }
}
```

List Affiliates

```
GET /v2/affiliates?status=active&limit=50&offset=0
```

Response (200):

```
{
  "success": true,
  "affiliates": [ /* array of affiliate objects */,
  "total": 25,
  "limit": 50,
  "offset": 0
}
```

Delete Affiliate

```
DELETE /v2/affiliates/{affiliate_id}
```

Response (200):

```
{
  "success": true,
  "message": "Affiliate deleted successfully"
}
```

Client Management

Create Client

```
POST /v2/clients
```

Request:

```
{
  "name": "Smith & Associates Law Firm",
  "code": "SMITH001",
  "status": "active",
  "contact_email": "leads@smithlaw.com",
  "contact_phone": "5559876543"
}
```

Response (201):

```
{
  "success": true,
  "client": {
    "id": "CLI12345678",
    "name": "Smith & Associates Law Firm",
    "code": "SMITH001",
    "status": "active",
    "created_at": "2026-02-05T10:00:00.000Z"
  }
}
```

```
}  
}
```

Update Client

```
PUT /v2/clients/{client_id}
```

Request:

```
{  
  "name": "Updated Client Name",  
  "status": "inactive"  
}
```

Response (200):

```
{  
  "success": true,  
  "client": { /* updated client object */ }  
}
```

Get Client

```
GET /v2/clients/{client_id}
```

Response (200):

```
{  
  "success": true,  
  "client": {  
    "id": "CLI12345678",  
    "name": "Smith & Associates Law Firm",  
    "code": "SMITH001",  
    "status": "active",  
    "campaigns": ["CAMP12345678"],  
    "total_leads_received": 450,  
    "purchase_rate": 93.3  
  }  
}
```

List Clients

```
GET /v2/clients?status=active&limit=50&offset=0
```

Response (200):

```
{  
  "success": true,  
  "clients": [ /* array of client objects */ ],  
  "total": 18,  
  "limit": 50,  
  "offset": 0  
}
```

Delete Client

```
DELETE /v2/clients/{client_id}
```

Response (200):

```
{
  "success": true,
  "message": "Client deleted successfully"
}
```

Get Campaign Fields

```
GET /v2/campaigns/{campaign_id}/fields
```

Purpose: Returns list of fields defined in the campaign's current criteria version. Used for building field mapping UI when linking affiliates.

Response (200):

```
{
  "success": true,
  "campaign_id": "CAMP12345678",
  "campaign_name": "Rideshare Abuse Campaign Q1 2026",
  "current_criteria_version": 1,
  "fields": [
    {
      "key": "first_name",
      "display_name": "First Name",
      "data_type": "string",
      "required": true,
      "description": "Lead's first name"
    },
    {
      "key": "last_name",
      "display_name": "Last Name",
      "data_type": "string",
      "required": true,
      "description": "Lead's last name"
    },
    {
      "key": "email",
      "display_name": "Email Address",
      "data_type": "string",
      "required": true,
      "description": "Lead's email address"
    },
    {
      "key": "phone",
      "display_name": "Phone Number",
      "data_type": "string",
      "required": true,
      "description": "Lead's phone number"
    },
    {
      "key": "state",
      "display_name": "State",
      "data_type": "string",
      "required": true,
      "description": "State where incident occurred"
    },
    {
      "key": "rideshare_abuse",
      "display_name": "Did you experience rideshare abuse?",
      "data_type": "boolean",
      "required": true,
      "description": "Indicates if the lead experienced abuse"
    }
  ],
}
```

```
{
  "key": "rideshare_company",
  "display_name": "Which rideshare company?",
  "data_type": "list",
  "required": true,
  "description": "The rideshare company involved",
  "options": [
    { "label": "Uber", "value": "uber" },
    { "label": "Lyft", "value": "lyft" },
    { "label": "Other", "value": "other" }
  ]
},
{
  "key": "has_attorney",
  "display_name": "Do you have an attorney?",
  "data_type": "boolean",
  "required": false,
  "description": "Whether the lead already has legal representation"
}
],
"standard_fields": [
  "first_name",
  "last_name",
  "email",
  "phone",
  "state",
  "ip_address",
  "trusted_form_cert"
],
"campaign_specific_fields": [
  "rideshare_abuse",
  "rideshare_company",
  "has_attorney"
]
}
```

Usage Example: When adding an affiliate to a campaign, the frontend: 1. Calls `GET /v2/campaigns/{campaign_id}/fields` 2. Displays the `fields` array in a dropdown for mapping configuration 3. User maps their custom fields to campaign fields: - `fname` → `first_name` - `lname` → `last_name` - `contact_email` → `email` - `phone_num` → `phone` 4. Submits `POST /v2/campaigns/{campaign_id}/affiliates` with `field_mappings`

Campaign-Affiliate Management

Affiliate Linking Workflow:

1. **Get Campaign Fields** - GET `/v2/campaigns/{campaign_id}/fields` to see available fields for mapping
2. **Configure Mappings** - Map affiliate's custom field names to campaign's expected fields:
 - Affiliate sends: `fname` , `lname` , `contact_email` , `phone_num`
 - Campaign expects: `first_name` , `last_name` , `email` , `phone`
 - Mappings: `{"fname": "first_name", "lname": "last_name", "contact_email": "email", "phone_num": "phone"}`
3. **Link Affiliate** - POST `/v2/campaigns/{campaign_id}/affiliates` with `field_mappings`
4. **Receive Campaign Key** - System generates unique `campaign_key` for this affiliate-campaign relationship
5. **Affiliate Integration** - Affiliate uses `campaign_key` in their lead submissions

Note: Field mappings are optional. If `custom_mapping_enabled: false` , affiliate must send fields using campaign's expected field names.

Link Affiliate to Campaign

`POST /v2/campaigns/{campaign_id}/affiliates`

Request:

```
{
  "affiliate_id": "AFF12345678",
  "status": "test", // Optional, defaults to "test" if not provided. Values: test, live, inactive
}
```

```
"cap_type": "lifetime", // Optional: daily, weekly, monthly, lifetime. Defaults to "lifetime"
"cap_limit": 100, // Maximum leads allowed for the cap period (required if cap_type provided)
"custom_mapping_enabled": true,
"field_mappings": {
  "fname": "first_name",
  "lname": "last_name",
  "contact_email": "email",
  "phone_num": "phone"
},
"postback_enabled": true,
"postback_url": "https://affiliate.com/postback"
}
```

Response (201):

```
{
  "success": true,
  "message": "Affiliate linked to campaign successfully",
  "affiliate_config": {
    "affiliate_id": "AFF12345678",
    "campaign_key": "CAMPKEY_ABC123XYZ789", // Auto-generated unique key for this affiliate-campaign relationship
    "status": "test",
    "added_at": "2026-02-05T10:30:00.000Z",
    "cap_type": "daily",
    "cap_limit": 100,
    "leads_sent_current_period": 0,
    "cap_period_start": "2026-02-05",
    "cap_period_end": "2026-02-06",
    "total_leads_sent": 0,
    "custom_mapping_enabled": true,
    "field_mappings": { /* mappings */ }
  },
  "instructions": "Affiliate is in TEST mode. Use campaign_key with /v2/lead/test endpoint. Change status to 'live' when ready"
}
```

Update Affiliate Configuration

```
PUT /v2/campaigns/{campaign_id}/affiliates/{affiliate_id}
```

Request:

```
{
  "status": "live", // test, live, inactive
  "cap_type": "weekly", // daily, weekly, monthly, lifetime
  "cap_limit": 500,
  "field_mappings": {
    "fname": "first_name",
    "email_address": "email"
  }
}
```

Response (200):

```
{
  "success": true,
  "message": "Affiliate configuration updated",
  "affiliate_config": { /* updated config */ }
}
```

Get Affiliate Capacity Status

GET /v2/campaigns/{campaign_id}/affiliates/{affiliate_id}/capacity

Response (200):

```
{
  "success": true,
  "affiliate_id": "AFF12345678",
  "campaign_id": "CAMP12345678",
  "status": "live",
  "cap_type": "weekly",
  "cap_limit": 500,
  "leads_sent_current_period": 245,
  "remaining_capacity": 255,
  "capacity_percentage": 49.0,
  "cap_period_start": "2026-02-03", // Monday of current week
  "cap_period_end": "2026-02-10", // Monday of next week
  "total_leads_sent": 1250, // All-time total
  "next_reset_time": "2026-02-10T00:00:00.000Z" // null for 'lifetime' cap type
}
```

Remove Affiliate from Campaign

DELETE /v2/campaigns/{campaign_id}/affiliates/{affiliate_id}

Request Body (optional):

```
{
  "soft_delete": true // Sets status to "deleted" but keeps in campaign.affiliates[]
}
```

Response (200):

```
{
  "success": true,
  "message": "Affiliate removed from campaign"
}
```

Campaign-Client Management

Client Linking Workflow:

- 1. **Link Client** - POST /v2/campaigns/{id}/clients with client_id
- 2. **Specify Config** - Choose to use campaign default criteria/logic OR specify custom version:
 - use_custom: false, criteria_version: null → Uses campaign's current_criteria_version
 - use_custom: true, criteria_version: 2 → Uses version 2 of campaign's criteria
- 3. **Validation** - System validates the specified version exists in Config Catalog
- 4. **Client Active** - Client receives leads validated against their specified criteria/logic version

Use Cases: - **Standard Client:** use_custom: false - Always uses campaign's latest/current version - **Custom Client:** use_custom: true, criteria_version: 2 - Locked to specific version (e.g., stricter requirements) - **Version Migration:** Update criteria_version to move client to newer validation rules

Link Client to Campaign

POST /v2/campaigns/{campaign_id}/clients

Request:

```
{
  "client_id": "CLI12345678",
  "status": "active",

  // Criteria Configuration
  "criteria_config": {
    "use_custom": false,           // If false, uses campaign's current_criteria_version
    "criteria_version": null      // null = use campaign default, or specify version number
  },

  // Logic Configuration
  "logic_config": {
    "use_custom": false,           // If false, uses campaign's current_logic_version
    "logic_version": null         // null = use campaign default, or specify version number
  },

  // Routing Configuration
  "routing_weight": 50,
  "routing_priority": 1,

  // Webhook Configuration
  "webhook_enabled": true,
  "webhook_url": "https://client.com/leads",
  "webhook_retry_attempts": 3
}
```

Response (201):

```
{
  "success": true,
  "message": "Client linked to campaign successfully",
  "client_config": {
    "client_id": "CLI12345678",
    "status": "active",
    "added_at": "2026-02-05T11:00:00.000Z",
    "criteria_config": {
      "use_custom": false,
      "criteria_version": null,
      "effective_version": 1          // The version that will actually be used (campaign's current)
    },
    "logic_config": {
      "use_custom": false,
      "logic_version": null,
      "effective_version": 1
    },
    "routing_weight": 50,
    "webhook_url": "https://client.com/leads"
  }
}
```

Update Client Configuration

```
PUT /v2/campaigns/{campaign_id}/clients/{client_id}
```

Request:

```
{
  "status": "active",
  "routing_weight": 30,
  "criteria_config": {
    "use_custom": true,           // Switch to custom criteria
    "criteria_version": 2        // Use version 2 of campaign criteria
  },
  "logic_config": {
```

```
"use_custom": false,           // Use campaign default
"logic_version": null
}
}
```

Response (200):

```
{
  "success": true,
  "message": "Client configuration updated",
  "client_config": { /* updated config */ }
}
```

Remove Client from Campaign

```
DELETE /v2/campaigns/{campaign_id}/clients/{client_id}
```

Request Body (optional):

```
{
  "soft_delete": true // Sets status to "deleted" but keeps in campaign.clients[]
}
```

Response (200):

```
{
  "success": true,
  "message": "Client removed from campaign"
}
```

Template Catalog Management

List Templates

```
GET /v2/templates?type={criteria|logic}&category={category}&limit=50
```

Response (200):

```
{
  "success": true,
  "templates": [
    {
      "id": "TPL12345678",
      "type": "criteria",
      "name": "Standard Rideshare Intake",
      "description": "Standard questions for rideshare abuse cases",
      "category": "rideshare",
      "is_system_template": true,
      "times_used": 25,
      "created_at": "2026-01-15T10:00:00.000Z"
    },
    {
      "id": "TPL87654321",
      "type": "logic",
      "name": "Rideshare Qualification Logic",
      "description": "Basic qualification rules for rideshare leads",
      "category": "rideshare",
      "is_system_template": true,
      "times_used": 25,

```

```
      "created_at": "2026-01-15T10:05:00.000Z"
    }
  ],
  "total": 2
}
```

Get Template Details

GET /v2/templates/{template_id}

Response (200):

```
{
  "success": true,
  "template": {
    "id": "TPL12345678",
    "type": "criteria",
    "name": "Standard Rideshare Intake",
    "description": "Standard questions for rideshare abuse cases",
    "category": "rideshare",
    "is_system_template": true,
    "config": {
      "questions": [
        {
          "key": "rideshare_abuse",
          "display_name": "Did you experience rideshare abuse?",
          "data_type": "boolean",
          "required": true
        }
        /* ... more questions */
      ]
    },
    "times_used": 25,
    "campaigns_using": 5
  }
}
```

Create Custom Template

POST /v2/templates

Request:

```
{
  "type": "criteria",
  "name": "Custom Auto Accident Intake",
  "description": "Custom questions for auto accident cases",
  "category": "auto_accident",
  "config": {
    "questions": [
      {
        "key": "accident_date",
        "display_name": "Date of accident",
        "data_type": "date",
        "required": true
      }
      /* ... more questions */
    ]
  }
}
```

Response (201):

```
{
  "success": true,
  "template": {
    "id": "TPL99887766",
    "type": "criteria",
    "name": "Custom Auto Accident Intake",
    "is_system_template": false,
    "tenant_id": "smashorbit",
    "created_at": "2026-02-05T11:00:00.000Z"
  }
}
```

Campaign Criteria Configuration

Get Campaign Criteria (Current Version)

```
GET /v2/campaigns/{campaign_id}/criteria
```

Response (200):

```
{
  "success": true,
  "campaign_id": "CAMP12345678",
  "version": 1,
  "status": "active",
  "created_at": "2026-02-01T10:00:00.000Z",
  "config": {
    "questions": [
      {
        "key": "rideshare_abuse",
        "display_name": "Did you experience rideshare abuse?",
        "data_type": "boolean",
        "description": "Indicates if the lead experienced abuse",
        "required": true
      }
      /* ... more questions */
    ]
  }
}
```

Get Criteria by Version

```
GET /v2/campaigns/{campaign_id}/criteria/versions/{version}
```

Response (200):

```
{
  "success": true,
  "campaign_id": "CAMP12345678",
  "version": 2,
  "config": { /* criteria config */ }
}
```

Create New Criteria Version

```
POST /v2/campaigns/{campaign_id}/criteria
```

Request:

```
{
  "questions": [
    {
      "key": "rideshare_abuse",
      "display_name": "Did you experience rideshare abuse?",
      "data_type": "boolean",
      "description": "Indicates if the lead experienced abuse from rideshare services",
      "required": true
    },
    {
      "key": "rideshare_company",
      "display_name": "Which rideshare company?",
      "data_type": "list",
      "description": "The rideshare company involved",
      "required": true,
      "options": [
        { "label": "Uber", "value": "uber" },
        { "label": "Lyft", "value": "lyft" },
        { "label": "Other", "value": "other" }
      ]
    },
    {
      "key": "state",
      "display_name": "State",
      "data_type": "string",
      "description": "State where incident occurred",
      "required": true
    }
  ]
}
```

Response (201):

```
{
  "success": true,
  "message": "New criteria version created",
  "version": 2,
  "campaign_id": "CAMP12345678",
  "config": { /* new criteria config */ }
}
```

Note: Creating a new version does NOT affect existing affiliates/clients using older versions.

Set Active Criteria Version

```
PUT /v2/campaigns/{campaign_id}/criteria/active
```

Request:

```
{
  "version": 2
}
```

Response (200):

```
{
  "success": true,
  "message": "Active criteria version updated",
  "current_criteria_version": 2
}
```

List All Criteria Versions

GET /v2/campaigns/{campaign_id}/criteria/versions

Response (200):

```
{
  "success": true,
  "campaign_id": "CAMP12345678",
  "versions": [
    {
      "version": 1,
      "status": "active",
      "created_at": "2026-02-01T10:00:00.000Z",
      "used_by_count": 5
    },
    {
      "version": 2,
      "status": "active",
      "created_at": "2026-02-04T15:30:00.000Z",
      "used_by_count": 2
    }
  ]
}
```

Campaign Logic Configuration

Get Campaign Logic (Current Version)

GET /v2/campaigns/{campaign_id}/logic

Response (200):

```
{
  "success": true,
  "campaign_id": "CAMP12345678",
  "version": 1,
  "status": "active",
  "config": {
    "rules": [
      {
        "name": "Rule 1: Rideshare abuse with no attorney",
        "groups": [
          {
            "operator": "AND",
            "conditions": [
              {
                "field": "rideshare_abuse",
                "operator": "==",
                "value": true
              },
              {
                "field": "has_attorney",
                "operator": "==",
                "value": false
              }
            ]
          }
        ],
        "action": "accept"
      }
    ],
    "combine_rules": "AND"
  }
}
```

```
}  
}
```

Create New Logic Version

POST /v2/campaigns/{campaign_id}/logic

Request:

```
{  
  "rules": [  
    {  
      "name": "Rule 1: Valid rideshare companies only",  
      "groups": [  
        {  
          "operator": "OR",  
          "conditions": [  
            {  
              "field": "rideshare_company",  
              "operator": "in",  
              "value": ["uber", "lyft"]  
            }  
          ]  
        }  
      ],  
      "action": "accept"  
    }  
  ],  
  "combine_rules": "AND"  
}
```

Response (201):

```
{  
  "success": true,  
  "message": "New logic version created",  
  "version": 2,  
  "campaign_id": "CAMP12345678",  
  "config": { /* new logic config */ }  
}
```

Set Active Logic Version

PUT /v2/campaigns/{campaign_id}/logic/active

Request:

```
{  
  "version": 2  
}
```

Response (200):

```
{  
  "success": true,  
  "message": "Active logic version updated",  
  "current_logic_version": 2  
}
```

List All Logic Versions

```
GET /v2/campaigns/{campaign_id}/logic/versions
```

Response (200):

```
{
  "success": true,
  "campaign_id": "CAMP12345678",
  "versions": [
    {
      "version": 1,
      "status": "active",
      "created_at": "2026-02-01T10:15:00.000Z",
      "used_by_count": 5
    },
    {
      "version": 2,
      "status": "active",
      "created_at": "2026-02-04T16:00:00.000Z",
      "used_by_count": 0
    }
  ]
}
```

Plugin Configuration

Get Campaign Plugins

```
GET /v2/campaigns/{campaign_id}/plugins
```

Response (200):

```
{
  "success": true,
  "plugins": {
    "ipqs": {
      "enabled": true,
      "config": { /* IPQS config */ }
    },
    "trustedform": {
      "enabled": true,
      "config": { /* TrustedForm config */ }
    },
    "duplicate_check": {
      "enabled": true,
      "config": { /* Duplicate check config */ }
    }
  }
}
```

Update Plugin Configuration

```
PUT /v2/campaigns/{campaign_id}/plugins/{plugin_name}
```

Request (IPQS example):

```
{
  "enabled": true,
  "config": {
```

```
"phone_enabled": true,
"email_enabled": true,
"ip_enabled": true,
"thresholds": {
  "phone": {
    "fraud_score": { "operator": "<", "value": 75 },
    "V0IP": { "operator": "==", "value": false },
    "valid": { "operator": "==", "value": true }
  },
  "email": {
    "fraud_score": { "operator": "<", "value": 85 },
    "valid": { "operator": "==", "value": true }
  }
}
}
```

Response (200):

```
{
  "success": true,
  "message": "Plugin configuration updated",
  "plugin": "ipqs",
  "config": { /* updated config */ }
}
```

Enable/Disable Plugin

```
PATCH /v2/campaigns/{campaign_id}/plugins/{plugin_name}
```

Request:

```
{
  "enabled": false
}
```

Response (200):

```
{
  "success": true,
  "message": "Plugin ipqs disabled"
}
```

Tenant Configuration

Get Tenant Plugin Credentials Status

```
GET /v2/tenant/plugins
```

Response (200):

```
{
  "success": true,
  "plugin_credentials": {
    "ipqs": {
      "configured": true,
      "configured_at": "2026-01-15T12:00:00.000Z"
    },
    "trustedform": {
```

```
    "configured": false,
    "configured_at": null
  }
}
```

Set Plugin Credentials

POST /v2/tenant/plugins/{plugin_name}/credentials

Request (IPQS example):

```
{
  "api_key": "ABC123XYZ789",
  "base_url": "https://ipqualityscore.com/api/json"
}
```

Response (200):

```
{
  "success": true,
  "message": "Plugin credentials saved successfully",
  "action_required": "Lambda redeployment triggered to load new credentials"
}
```

Note: This endpoint stores credentials in Secrets Manager and triggers a Lambda redeployment to update environment variables.

Update Plugin Credentials

PUT /v2/tenant/plugins/{plugin_name}/credentials

Request: Same as POST

Response (200):

```
{
  "success": true,
  "message": "Plugin credentials updated successfully",
  "action_required": "Lambda redeployment triggered to reload credentials"
}
```

Update Notification Emails

PUT /v2/tenant/notifications

Request:

```
{
  "leads": "leads@smashorbit.com",
  "alerts": "alerts@smashorbit.com",
  "admin": "admin@smashorbit.com",
  "unauthorized_access": true, // Enable notifications when test/inactive affiliates try production endpoint
  "capacity_warnings": true, // Enable notifications when affiliates reach 80%, 90%, 100% of cap
  "capacity_threshold": 80 // Percentage at which to send first warning (default: 80)
}
```

Response (200):

```
{
  "success": true,
  "message": "Notification emails updated"
}
```

Notification Types Sent:

1. **Unauthorized Access Alert** (when enabled):
- Sent when affiliate with status=“test” or “inactive” attempts to POST to `/v2/lead/intake`
 - Includes: affiliate_id, campaign_id, affiliate status, timestamp, IP address
 - Sent to: `alerts` email
2. **Capacity Warning** (when enabled):
- Sent when affiliate reaches configured threshold (default 80%)
 - Sent when affiliate reaches 90% capacity
 - Sent when affiliate reaches 100% capacity (cap hit)
 - Works with all cap types: daily, weekly, monthly, lifetime
 - Includes: affiliate_id, campaign_id, cap_type, current count, cap limit, percentage, period dates
 - Sent to: `admin` email

Analytics & Reporting

Get Lead Statistics

```
GET /v2/analytics/leads?campaign_id={id}&start_date={date}&end_date={date}
```

Response (200):

```
{
  "success": true,
  "campaign_id": "CAMP12345678",
  "date_range": {
    "start": "2026-02-01",
    "end": "2026-02-05"
  },
  "stats": {
    "total_leads": 1250,
    "accepted": 980,
    "rejected": 270,
    "acceptance_rate": 78.4,
    "rejection_reasons": {
      "duplicate": 45,
      "ipqs_failed": 125,
      "trustedform_failed": 85,
      "criteria_failed": 15
    }
  }
}
```

Get Affiliate Performance

```
GET /v2/analytics/affiliates/{affiliate_id}?start_date={date}&end_date={date}
```

Response (200):

```
{
  "success": true,
  "affiliate_id": "AFF12345678",
  "stats": {
    "total_leads_sent": 450,
    "accepted": 380,
    "rejected": 70,
    "acceptance_rate": 84.4,
    "campaigns": [
      {
        "campaign_id": "CAMP12345678",
        "campaign_name": "Rideshare Q1",
        "leads_sent": 450,
        "acceptance_rate": 84.4
      }
    ]
  }
}
```

Get Client Performance

```
GET /v2/analytics/clients/{client_id}?start_date={date}&end_date={date}
```

Response (200):

```
{
  "success": true,
  "client_id": "CLI12345678",
  "stats": {
    "total_leads_received": 420,
    "leads_purchased": 400,
    "purchase_rate": 95.2,
    "average_lead_price": 50.00,
    "total_revenue": 20000.00
  }
}
```

Query Raw Lead Data (S3/Athena)

```
POST /v2/analytics/query
```

Request:

```
{
  "query_type": "raw_payloads",
  "filters": {
    "campaign_id": "CAMP12345678",
    "start_date": "2026-02-01",
    "end_date": "2026-02-05",
    "remapped": true
  },
  "limit": 100
}
```

Response (200):

```
{
  "success": true,
  "query_id": "query-abc123",
}
```

```
"status": "running",
"message": "Athena query started, use GET /v2/analytics/query/{query_id} to check status"
}
```

Get Query Results

GET /v2/analytics/query/{query_id}

Response (200):

```
{
  "success": true,
  "query_id": "query-abc123",
  "status": "completed",
  "results": [
    {
      "lead_id": "LEAD12345678",
      "timestamp": "2026-02-04T12:30:45.123Z",
      "raw_payload": { /* original affiliate payload */ },
      "field_mappings": { /* mapping applied */ },
      "transformed_payload": { /* transformed payload */ }
    }
    /* ... more results */
  ],
  "count": 45
}
```

Lead Management

Get Lead Details

GET /v2/leads/{lead_id}

Response (200):

```
{
  "success": true,
  "lead": {
    "id": "LEAD12345678",
    "campaign_id": "CAMP12345678",
    "first_name": "John",
    "last_name": "Doe",
    "email": "john@example.com",
    "phone": "5551234567",
    // ... all lead fields
    "cherry_pick_enabled": true,
    "cherry_picked": false,
    "edited": true,
    "edit_count": 2,
    "edited_fields": ["phone", "email"],
    "last_edited_by": "user@smashorbit.com",
    "last_edited_at": "2026-02-05T14:30:00.000Z"
  },
  "campaign": {
    "id": "CAMP12345678",
    "name": "Rideshare Abuse Campaign Q1 2026",
    "lead_edit_config": {
      "enabled": true,
      "require_reason": true
    }
  },
  "editable_fields": [
    // Fields user can edit based on campaign config + user role
    "first_name", "last_name", "email", "phone", "state", "rideshare_company"
  ]
}
```

```
],
"readonly_fields": [
  "id", "campaign_id", "affiliate_id", "created_at", "ip_address"
],
"field_metadata": {
  "phone": {
    "edited": true,
    "last_edit": "2026-02-05T14:30:00.000Z",
    "edit_count": 1
  },
  "email": {
    "edited": true,
    "last_edit": "2026-02-05T14:15:00.000Z",
    "edit_count": 1
  }
}
```

Update Lead

PUT /v2/leads/{lead_id}

Request:

```
{
  "changes": [
    {
      "field_name": "phone",
      "new_value": "5559876543",
      "change_reason": "Customer requested update via email"
    },
    {
      "field_name": "email",
      "new_value": "newemail@example.com",
      "change_reason": "Typo correction"
    }
  ]
}
```

Response (200):

```
{
  "success": true,
  "message": "Lead updated successfully",
  "lead_id": "LEAD12345678",
  "edit_id": "EDIT12345678",
  "changes_applied": 2,
  "updated_fields": ["phone", "email"],
  "updated_at": "2026-02-05T15:00:00.000Z",
  "updated_by": "user@smashorbit.com"
}
```

Error Response (403):

```
{
  "success": false,
  "error": "Field not editable",
  "message": "The field 'campaign_id' is marked as readonly in this campaign's configuration and cannot be edited",
  "campaign_id": "CAMP12345678",
  "readonly_fields": ["id", "campaign_id", "affiliate_id", "created_at", "ip_address", "trusted_form_cert"]
}
```

Error Response (403 - Campaign editing disabled):

```
{
  "success": false,
  "error": "Lead editing disabled",
  "message": "Lead editing is disabled for this campaign",
  "campaign_id": "CAMP12345678"
}
```

Get Lead Edit History

```
GET /v2/leads/{lead_id}/history
```

Response (200):

```
{
  "success": true,
  "lead_id": "LEAD12345678",
  "total_edits": 3,
  "history": [
    {
      "edit_id": "EDIT12345678",
      "timestamp": "2026-02-05T15:00:00.000Z",
      "edited_by": "user@smashorbit.com",
      "edited_by_name": "John Doe",
      "edit_type": "field_update",
      "changes": [
        {
          "field_name": "phone",
          "field_display_name": "Phone Number",
          "old_value": "5551234567",
          "new_value": "5559876543",
          "change_reason": "Customer requested update"
        }
      ]
    },
    {
      "edit_id": "EDIT12345677",
      "timestamp": "2026-02-05T14:15:00.000Z",
      "edited_by": "admin@smashorbit.com",
      "edited_by_name": "Admin User",
      "edit_type": "field_update",
      "changes": [
        {
          "field_name": "email",
          "field_display_name": "Email Address",
          "old_value": "old@example.com",
          "new_value": "john@example.com",
          "change_reason": "Typo correction"
        }
      ]
    },
    {
      "edit_id": "EDIT12345676",
      "timestamp": "2026-02-04T16:00:00.000Z",
      "edited_by": "manager@smashorbit.com",
      "edited_by_name": "Manager User",
      "edit_type": "cherry_pick",
      "changes": [
        {
          "field_name": "cherry_picked",
          "old_value": false,
          "new_value": true,
          "change_reason": "High-value lead selected for premium client"
        }
      ]
    }
  ]
}
```

```
}  
]  
}
```

Toggle Cherry Pick

PATCH /v2/leads/{lead_id}/cherry-pick

Request:

```
{  
  "cherry_picked": true,  
  "reason": "High-value lead selected for premium client"  
}
```

Response (200):

```
{  
  "success": true,  
  "message": "Lead cherry pick status updated",  
  "lead_id": "LEAD12345678",  
  "cherry_picked": true,  
  "cherry_picked_by": "user@smashorbit.com",  
  "cherry_picked_at": "2026-02-05T15:30:00.000Z"  
}
```

Error Response (403):

```
{  
  "success": false,  
  "error": "Cherry pick disabled",  
  "message": "This lead does not have cherry_pick_enabled set to true"  
}
```

Update Campaign Lead Edit Configuration

PUT /v2/campaigns/{campaign_id}/lead-edit-config

Request:

```
{  
  "enabled": true,  
  "require_reason": true,  
  "editable_fields": [  
    "first_name",  
    "last_name",  
    "email",  
    "phone",  
    "state",  
    "rideshare_company",  
    "has_attorney"  
  ],  
  "readonly_fields": [  
    "id",  
    "timestamp",  
    "campaign_id",  
    "affiliate_id",  
    "client_id",  
  ]  
}
```

```
    "created_at",
    "ip_address",
    "trusted_form_cert",
    "quality_checks"
  ]
}
```

Response (200):

```
{
  "success": true,
  "message": "Campaign lead edit configuration updated",
  "campaign_id": "CAMP12345678",
  "lead_edit_config": {
    "enabled": true,
    "require_reason": true,
    "editable_fields": [ /* updated list */ ],
    "readonly_fields": [ /* updated list */ ]
  }
}
```

Note: This allows each campaign to have its own editable field configuration. For example: - Campaign A (Rideshare): Allow editing `rideshare_company` , `has_attorney` - Campaign B (Auto Accident): Allow editing `accident_date` , `injury_type` - Campaign C (Medical): More restricted - only allow name/contact updates

User Management

Create User

POST `/v2/users`

Request:

```
{
  "email": "newuser@smashorbit.com",
  "first_name": "Jane",
  "last_name": "Smith",
  "password": "SecurePassword123!",
  "role": "editor",
  "permissions": [
    "leads.view",
    "leads.edit"
  ]
}
```

Response (201):

```
{
  "success": true,
  "user": {
    "id": "USER87654321",
    "email": "newuser@smashorbit.com",
    "display_name": "Jane Smith",
    "role": "editor",
    "status": "active",
    "created_at": "2026-02-05T16:00:00.000Z"
  }
}
```

List Users

```
GET /v2/users?role={role}&status={status}
```

Response (200):

```
{
  "success": true,
  "users": [
    {
      "id": "USER12345678",
      "email": "admin@smashorbit.com",
      "display_name": "Admin User",
      "role": "admin",
      "status": "active",
      "last_login_at": "2026-02-05T10:00:00.000Z"
    }
  ],
  "total": 5
}
```

Update User

```
PUT /v2/users/{user_id}
```

Request:

```
{
  "role": "manager",
  "permissions": [
    "leads.view",
    "leads.edit",
    "leads.cherry_pick"
  ],
  "status": "active"
}
```

Response (200):

```
{
  "success": true,
  "message": "User updated successfully",
  "user": { /* updated user object */ }
}
```

Authentication Endpoints

Login

```
POST /v2/auth/login
```

Request:

```
{
  "email": "user@smashorbit.com",
  "password": "SecurePassword123!"
}
```

Response (200):

```
{
  "success": true,
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "expires_at": "2026-02-06T16:00:00.000Z",
  "user": {
    "id": "USER12345678",
    "email": "user@smashorbit.com",
    "display_name": "John Doe",
    "role": "admin",
    "permissions": ["leads.view", "leads.edit", "campaigns.manage"]
  }
}
```

Logout

POST /v2/auth/logout

Request: Requires Authorization header with JWT token

Response (200):

```
{
  "success": true,
  "message": "Logged out successfully"
}
```

Webhook Formats

Client Webhook

Method: POST
URL: Configured per client in campaign
Timeout: 30 seconds
Retry: 3 attempts with exponential backoff
Payload:

```
{
  "lead_id": "LEAD12345678",
  "timestamp": "2026-02-04T12:30:45.123Z",
  "campaign_id": "CAMP12345678",
  "campaign_name": "Rideshare Abuse Campaign Q1 2026",

  "first_name": "John",
  "last_name": "Doe",
  "email": "john@example.com",
  "phone": "1234567890",
  "state": "TX",
  "ip_address": "1.2.3.4",

  "rideshare_abuse": true,
  "rideshare_company": "uber",
  "has_attorney": false,

  "quality_checks": {
    "ipqs": {
      "passed": true,
      "fraud_scores": {
        "phone": 15,
        "email": 10,
        "ip": 5
      }
    }
  },
  "trustedform": {
```

```
    "passed": true,
    "cert_id": "abc123def456"
  }
},

"price": 50.00,
"currency": "USD"
}
```

Expected Response (200):

```
{
  "success": true,
  "reference_id": "CLIENT-REF-123"
}
```

Affiliate Postback

Method: GET or POST (configurable)
URL: Configured per affiliate in campaign

GET Example:

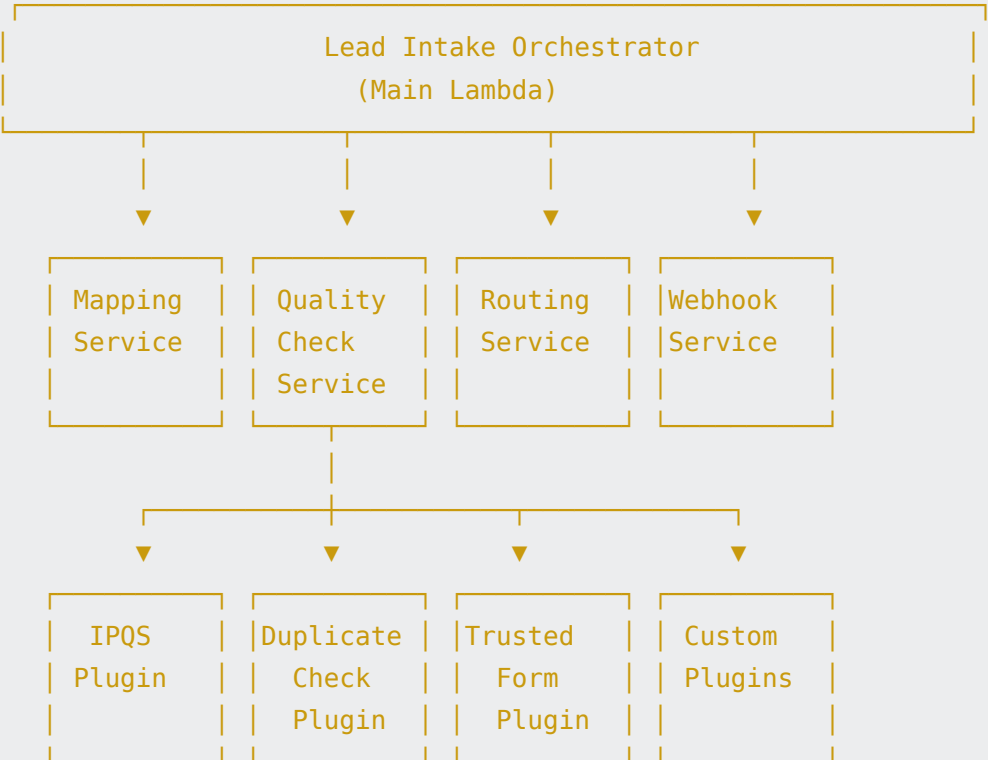
https://affiliate.com/postback?lead_id=LEAD12345678&status=accepted×tamp=2026-02-04T12:30:45.123Z

POST Payload:

```
{
  "lead_id": "LEAD12345678",
  "status": "accepted", // or "rejected"
  "timestamp": "2026-02-04T12:30:45.123Z",
  "rejection_reason": null, // or specific reason if rejected
  "details": {
    "sold": true,
    "client_name": "Smith & Associates Law Firm"
  }
}
```

Core Services

Service Architecture



1. Mapping Service

Purpose: Transform affiliate payloads to campaign schema

Interface:

```
interface MappingService {
  applyMapping(
    payload: Record<string, any>,
    fieldMappings: Record<string, string>
  ): MappedResult;
}

interface MappedResult {
  transformedPayload: Record<string, any>;
  fieldMappings: Record<string, string>;
  remapped: boolean;
}
```

Implementation: - Located in: `services/mappingService.ts` - Pure function (no side effects) - Validates mapped fields exist in campaign criteria

2. Quality Check Service

Purpose: Plugin orchestrator for validation checks

Interface:

```
interface QualityCheckService {
  executeChecks(
    lead: Lead,
    plugins: PluginConfig[],
    tenantId: string
  ): Promise<QualityCheckResults>;
}

interface PluginConfig {
  name: string;
  enabled: boolean;
  config: Record<string, any>;
}

interface QualityCheckResults {
  [pluginName: string]: {
    enabled: boolean;
    passed: boolean;
    results: any;
  };
}
```

Plugins:

a) IPQS Plugin

```
interface IpqsPlugin {
  validate(
    ipAddress: string,
    phone: string,
    email: string,
    thresholds: IpqsThresholds,
    credentials: IpqsCredentials
  ): Promise<IpqsResult>;
}
```

Location: `services/plugins/ipqsPlugin.ts`

b) Duplicate Check Plugin

```
interface DuplicateCheckPlugin {
  check(
    phone: string,
    email: string,
    config: DuplicateCheckConfig
  ): Promise<DuplicateCheckResult>;
}

interface DuplicateCheckConfig {
  phone_enabled: boolean;
  email_enabled: boolean;
  operator: 'AND' | 'OR';
  action: 'reject' | 'flag' | 'allow';
}
```

Location: services/plugins/duplicateCheckPlugin.ts

c) TrustedForm Plugin

```
interface TrustedFormPlugin {
  validate(
    certId: string,
    phone: string,
    credentials: TrustedFormCredentials
  ): Promise<TrustedFormResult>;
}
```

Location: services/plugins/trustedFormPlugin.ts

3. Routing Service

Purpose: Select client for lead based on routing rules

Interface:

```
interface RoutingService {
  selectClient(
    qualifiedClients: ClientConfig[],
    routingType: RoutingType,
    routingState: RoutingState
  ): ClientConfig | null;
}

type RoutingType = 'round_robin' | 'weighted' | 'priority' | 'traffic_based';

interface RoutingState {
  last_client_index?: number;
  client_counts?: Record<string, number>;
}
```

Algorithms: - **Round Robin**: Rotate through clients sequentially - **Weighted**: Distribute based on percentage/ratio weights - **Priority**: Highest priority (lowest number) first - **Traffic Based**: Custom logic based on current traffic patterns

Location: services/routingService.ts

4. Webhook Service

Purpose: Deliver webhooks to clients and postbacks to affiliates

Interface:

```
interface WebhookService {
  sendClientWebhook(
    lead: Lead,
    client: ClientConfig,
    retryAttempts: number
  ): Promise<WebhookResult>;

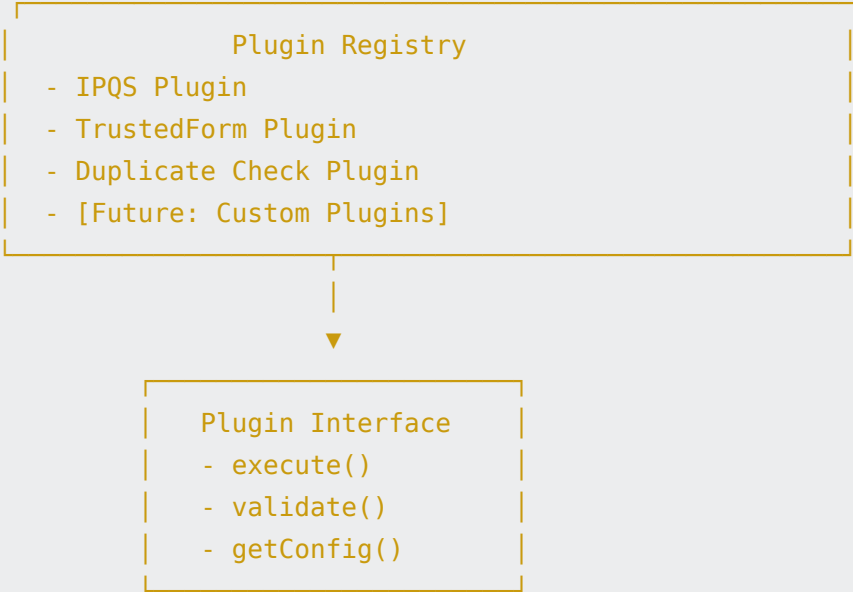
  sendAffiliatePostback(
    lead: Lead,
    affiliate: AffiliateConfig
  ): Promise<PostbackResult>;
}
```

Implementation: - SQS-based async delivery - Exponential backoff retry (1s, 2s, 4s) - DLQ for failed webhooks after max retries - Update lead record with webhook status

Location: `services/webhookService.ts`

Quality Checks & Plugins

Plugin System Architecture



Plugin Configuration Schema

Each plugin in `campaign.plugins` follows this structure:

```
{
  "plugin_name": {
    "enabled": true,
    "config": {
      // Plugin-specific configuration
    }
  }
}
```

IPQS Plugin Configuration

Campaign-Level Config:

```
{
  "ipqs": {
    "enabled": true,
    "config": {
      "phone_enabled": true,
      "email_enabled": true,
      "ip_enabled": true,
      "thresholds": {
```

```
    "phone": {
      "fraud_score": { "operator": "<", "value": 75 },
      "VOIP": { "operator": "==", "value": false },
      "valid": { "operator": "==", "value": true },
      "risky": { "operator": "==", "value": false },
      "active": { "operator": "==", "value": true }
    },
    "email": {
      "fraud_score": { "operator": "<", "value": 75 },
      "valid": { "operator": "==", "value": true },
      "disposable": { "operator": "==", "value": false },
      "spam_trap_score": { "operator": "==", "value": "none" }
    },
    "ip": {
      "fraud_score": { "operator": "<", "value": 75 },
      "proxy": { "operator": "==", "value": false },
      "vpn": { "operator": "==", "value": false },
      "tor": { "operator": "==", "value": false },
      "recent_abuse": { "operator": "==", "value": false }
    }
  }
}
```

Supported Operators: - **==** : Equal - **!=** : Not equal - **<** : Less than - **<=** : Less than or equal - **>** : Greater than - **>=** : Greater than or equal - **in** : Value in array - **contains** : String contains

TrustedForm Plugin Configuration

```
{
  "trustedform": {
    "enabled": true,
    "config": {
      "require_valid_cert": true,
      "cert_age_max_hours": 24
    }
  }
}
```

Duplicate Check Plugin Configuration

```
{
  "duplicate_check": {
    "enabled": true,
    "config": {
      "phone_enabled": true,
      "email_enabled": true,
      "operator": "OR",
      "action": "reject",
      "lookback_days": 30
    }
  }
}
```

Operator Behavior: - **OR** : Reject if phone OR email exists (more strict) - **AND** : Reject only if BOTH phone AND email match (less strict)

Action Behavior: - **reject** : Throw error, lead not saved (current behavior) - **flag** : Continue processing, mark lead as duplicate - **allow** : Continue processing, log duplicate in results

Account-Level Plugin Credentials

Stored in AWS Secrets Manager:

```
// Secret: {tenant}-{system}-ipqs-credentials
{
  "api_key": "ABC123XYZ789",
  "base_url": "https://ipqualityscore.com/api/json"
}

// Secret: {tenant}-{system}-trustedform-credentials
{
  "username": "API",
  "password": "TF_API_KEY_HERE"
}
```

Credential Management Flow:

1. Initial Setup (First-time plugin enablement):

- User enables plugin in campaign UI
- System checks `tenant_config.plugin_credentials[plugin_name].configured`
- If not configured, prompt for credentials via UI
- Store credentials in Secrets Manager
- Update `tenant_config` with secret ARN and `configured: true`
- **Trigger Lambda redeployment** to load credentials into environment variables

2. Credential Updates (When credentials change):

- Admin updates credentials via UI
- Update secret in Secrets Manager
- **Trigger Lambda redeployment** to refresh environment variables
- No code changes needed, just redeploy

3. Runtime Usage (Lead processing):

- Lambda reads credentials from environment variables (set at deployment)
- No Secrets Manager API calls during lead processing
- Zero latency overhead, no additional API costs

CDK Stack Integration:

```
// In servicesStack.ts
const ipqsSecret = secretsmanager.Secret.fromSecretNameV2(
  this,
  'IpqsSecret',
  `${tenantId}-${system}-ipqs-credentials`
);

const trustedformSecret = secretsmanager.Secret.fromSecretNameV2(
  this,
  'TrustedFormSecret',
  `${tenantId}-${system}-trustedform-credentials`
);

const leadIntakeLambda = new lambda.Function(this, 'LeadIntakeLambda', {
  // ... other config
  environment: {
    // Load from Secrets Manager at deployment time
    IPQS_API_KEY: ipqsSecret.secretValueFromJson('api_key').toString(),
    IPQS_BASE_URL: ipqsSecret.secretValueFromJson('base_url').toString(),
    TRUSTEDFORM_USERNAME: trustedformSecret.secretValueFromJson('username').toString(),
    TRUSTEDFORM_PASSWORD: trustedformSecret.secretValueFromJson('password').toString(),
  }
});

// Grant read permissions (only used during deployment, not runtime)
ipqsSecret.grantRead(leadIntakeLambda);
trustedformSecret.grantRead(leadIntakeLambda);
```

Benefits: -  No Secrets Manager API calls during lead processing (~50-100ms saved per lead) -  No additional AWS costs for secret retrieval - 
Credentials cached in Lambda environment for fast access -  Secrets Manager still provides secure storage and audit trail -  Simple redeployment updates credentials across all Lambda instances

Routing & Distribution

Routing Types

1. Round Robin

Description: Distribute leads evenly across qualified clients

Algorithm:

```
function roundRobinRouting(qualifiedClients, state) {
  const index = state.last_client_index || 0;
  const selectedClient = qualifiedClients[index % qualifiedClients.length];

  // Update state
  state.last_client_index = (index + 1) % qualifiedClients.length;

  return selectedClient;
}
```

Example: 3 clients (A, B, C) - Lead 1 → Client A - Lead 2 → Client B - Lead 3 → Client C - Lead 4 → Client A (cycle repeats)

2. Weighted Distribution

Description: Distribute based on percentage or ratio weights

Configuration:

```
{
  "clients": [
    { "client_id": "CLI11111111", "routing_weight": 50 }, // 50%
    { "client_id": "CLI22222222", "routing_weight": 30 }, // 30%
    { "client_id": "CLI33333333", "routing_weight": 20 }  // 20%
  ]
}
```

Algorithm:

```
function weightedRouting(qualifiedClients) {
  // Calculate total weight
  const totalWeight = qualifiedClients.reduce((sum, c) => sum + c.routing_weight, 0);

  // Generate random number [0, totalWeight)
  let random = Math.random() * totalWeight;

  // Select client
  for (const client of qualifiedClients) {
    random -= client.routing_weight;
    if (random <= 0) {
      return client;
    }
  }

  return qualifiedClients[0]; // Fallback
}
```

Supports Both: - **Percentages:** Weights sum to 100 (50, 30, 20) - **Ratios:** Weights can be any values (5, 3, 2 = same as 50%, 30%, 20%)

3. Priority-Based

Description: Route to highest priority client first

Configuration:

```
{
  "clients": [
    { "client_id": "CLI11111111", "routing_priority": 1 }, // Highest
    { "client_id": "CLI22222222", "routing_priority": 2 },
    { "client_id": "CLI33333333", "routing_priority": 3 } // Lowest
  ]
}
```

Algorithm:

```
function priorityRouting(qualifiedClients) {
  // Sort by priority (1 = highest)
  const sorted = qualifiedClients.sort((a, b) =>
    a.routing_priority - b.routing_priority
  );

  return sorted[0];
}
```

4. Traffic-Based

Description: Custom logic based on current traffic and client counts

Algorithm:

```
function trafficBasedRouting(qualifiedClients, state) {
  // Get current counts for each client
  const counts = state.client_counts || {};

  // Find client with lowest count
  let minCount = Infinity;
  let selectedClient = null;

  for (const client of qualifiedClients) {
    const count = counts[client.client_id] || 0;
    if (count < minCount) {
      minCount = count;
      selectedClient = client;
    }
  }

  return selectedClient || qualifiedClients[0];
}
```

Lead Caps

Affiliate Daily Caps

```
// Stored in campaign.affiliates[]
{
  "affiliate_id": "AFF12345678",
  "daily_cap": 100,
  "current_daily_count": 45,
  "last_reset_date": "2026-02-04"
}

// Check cap before processing
async function checkAffiliateCap(affiliate) {
  const today = new Date().toISOString().split('T')[0];
```

```
// Reset counter if new day
if (affiliate.last_reset_date !== today) {
  affiliate.current_daily_count = 0;
  affiliate.last_reset_date = today;
}

// Check cap
if (affiliate.daily_cap && affiliate.current_daily_count >= affiliate.daily_cap) {
  throw new Error(`Affiliate daily cap reached (${affiliate.daily_cap} leads per day)`);
}

// Increment counter
affiliate.current_daily_count++;
}
```

Future: Client Caps

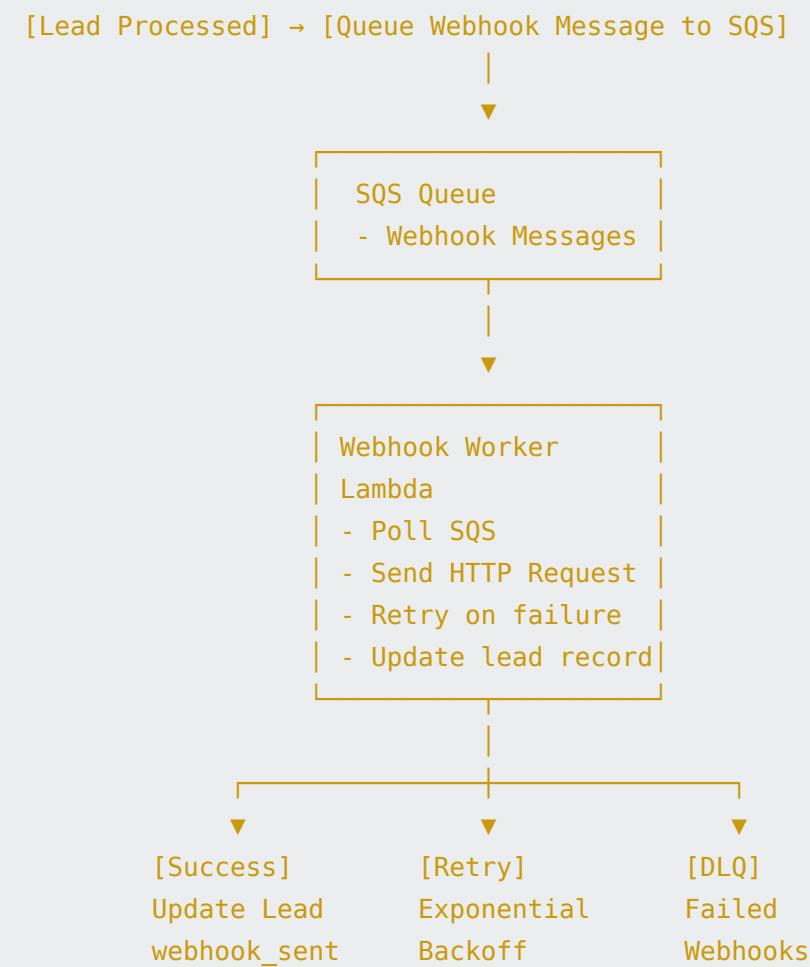
Design Considerations: - Daily cap per client - Monthly cap per client - Cap per campaign - Cap per affiliate-client relationship - Configurable cap reset time (midnight UTC, midnight local, etc.)

Proposed Schema:

```
{
  "clients": [
    {
      "client_id": "CLI12345678",
      "caps": {
        "daily": 50,
        "monthly": 1000,
        "campaign_total": null,
        "reset_time": "00:00:00",
        "reset_timezone": "America/Chicago"
      }
    }
  ]
}
```

Webhooks & Notifications

Webhook Delivery Architecture



Client Webhook Delivery

SQS Message Format:

```
{
  "type": "client_webhook",
  "lead_id": "LEAD12345678",
  "client_id": "CLI12345678",
  "webhook_url": "https://client.com/leads",
  "retry_attempts": 3,
  "attempt_number": 1,
  "payload": {
    "lead_id": "LEAD12345678",
    "timestamp": "2026-02-04T12:30:45.123Z",
    // ... full lead data
  }
}
```

Webhook Worker Lambda:

```
async function processWebhook(message) {
  const { webhook_url, payload, retry_attempts, attempt_number } = message;

  try {
    // Send webhook
    const response = await axios.post(webhook_url, payload, {
      timeout: 30000, // 30 second timeout
      headers: {
        'Content-Type': 'application/json',
        'X-Webhook-Signature': generateSignature(payload)
      }
    });

    // Update lead record
    await leadsTable.update({
      id: message.lead_id
    }, {
      webhook_sent: true,
      webhook_response_code: response.status,
      webhook_sent_at: new Date().toISOString()
    });

    return { success: true };
  } catch (error) {
    // Log error
    console.error('Webhook delivery failed:', error);

    // Retry with exponential backoff
    if (attempt_number < retry_attempts) {
      const delay = Math.pow(2, attempt_number) * 1000; // 1s, 2s, 4s

      await sqsClient.sendMessage({
        QueueUrl: process.env.WEBHOOK_QUEUE_URL,
        MessageBody: JSON.stringify({
          ...message,
          attempt_number: attempt_number + 1
        }),
        DelaySeconds: Math.min(delay / 1000, 900) // Max 15 minutes
      });

      return { success: false, retrying: true };
    }
  }
}
```

```
// Max retries exceeded, send to DLQ
await sqsClient.sendMessage({
  QueueUrl: process.env.WEBHOOK_DLQ_URL,
  MessageBody: JSON.stringify({
    ...message,
    error: error.message,
    failed_at: new Date().toISOString()
  })
});

// Update lead record
await leadsTable.update({
  id: message.lead_id
}, {
  webhook_sent: false,
  webhook_error: error.message
});

return { success: false, retrying: false };
}
```

Webhook Security

Signature Generation:

```
function generateSignature(payload, secret) {
  const hmac = crypto.createHmac('sha256', secret);
  hmac.update(JSON.stringify(payload));
  return hmac.digest('hex');
}
```

Client-Side Verification:

```
function verifyWebhook(payload, signature, secret) {
  const expectedSignature = generateSignature(payload, secret);
  return crypto.timingSafeEqual(
    Buffer.from(signature),
    Buffer.from(expectedSignature)
  );
}
```

Affiliate Postback Delivery

SQS Message Format:

```
{
  "type": "affiliate_postback",
  "lead_id": "LEAD12345678",
  "affiliate_id": "AFF12345678",
  "postback_url": "https://affiliate.com/postback",
  "payload": {
    "lead_id": "LEAD12345678",
    "status": "accepted",
    "rejection_reason": null,
    "timestamp": "2026-02-04T12:30:45.123Z"
  }
}
```

Processing: Same as client webhook with retry logic

Email Notifications

Email Templates:

1. Lead Sold Template

```
<!DOCTYPE html>
<html>
<head>
  <style>
    body { font-family: Arial, sans-serif; background-color: #f4f4f4; padding: 20px; }
    .container { background-color: white; padding: 30px; border-radius: 10px; }
    .status-sold { color: #28a745; font-weight: bold; font-size: 24px; }
    table { border-collapse: collapse; width: 100%; margin-top: 20px; }
    th, td { border: 1px solid #ddd; padding: 12px; text-align: left; }
    th { background-color: #28a745; color: white; }
    .success { color: #28a745; }
    .failed { color: #dc3545; }
  </style>
</head>
<body>
  <div class="container">
    <div class="status-sold">✔ LEAD SOLD</div>

    <h2>Lead Information</h2>
    <table>
      <tr><th>Field</th><th>Value</th></tr>
      <tr><td>Lead ID</td><td>{{lead_id}}</td></tr>
      <tr><td>Campaign</td><td>{{campaign_name}}</td></tr>
      <tr><td>Client</td><td>{{client_name}}</td></tr>
      <tr><td>Price</td><td>${{price}}</td></tr>
      <tr><td>Timestamp</td><td>{{timestamp}}</td></tr>
    </table>

    <h2>Contact Details</h2>
    <table>
      <tr><td>Name</td><td>{{first_name}} {{last_name}}</td></tr>
      <tr><td>Email</td><td>{{email}}</td></tr>
      <tr><td>Phone</td><td>{{phone}}</td></tr>
      <tr><td>State</td><td>{{state}}</td></tr>
    </table>

    <h2>Quality Checks</h2>
    <table>
      <tr>
        <td>IPQS</td>
        <td class="{{ipqs_class}}">{{ipqs_status}}</td>
      </tr>
      <tr>
        <td>TrustedForm</td>
        <td class="{{tf_class}}">{{tf_status}}</td>
      </tr>
      <tr>
        <td>Duplicate Check</td>
        <td class="{{dup_class}}">{{dup_status}}</td>
      </tr>
    </table>

    <h2>Criteria Validation</h2>
    <table>
      <tr>
        <td>Criteria (v{{criteria_version}})</td>
        <td class="{{criteria_class}}">{{criteria_status}}</td>
      </tr>
      <tr>
        <td>Logic (v{{logic_version}})</td>
        <td class="{{logic_class}}">{{logic_status}}</td>
      </tr>
    </table>
  </div>
```

```
</body>
</html>
```

2. Lead Rejected Template

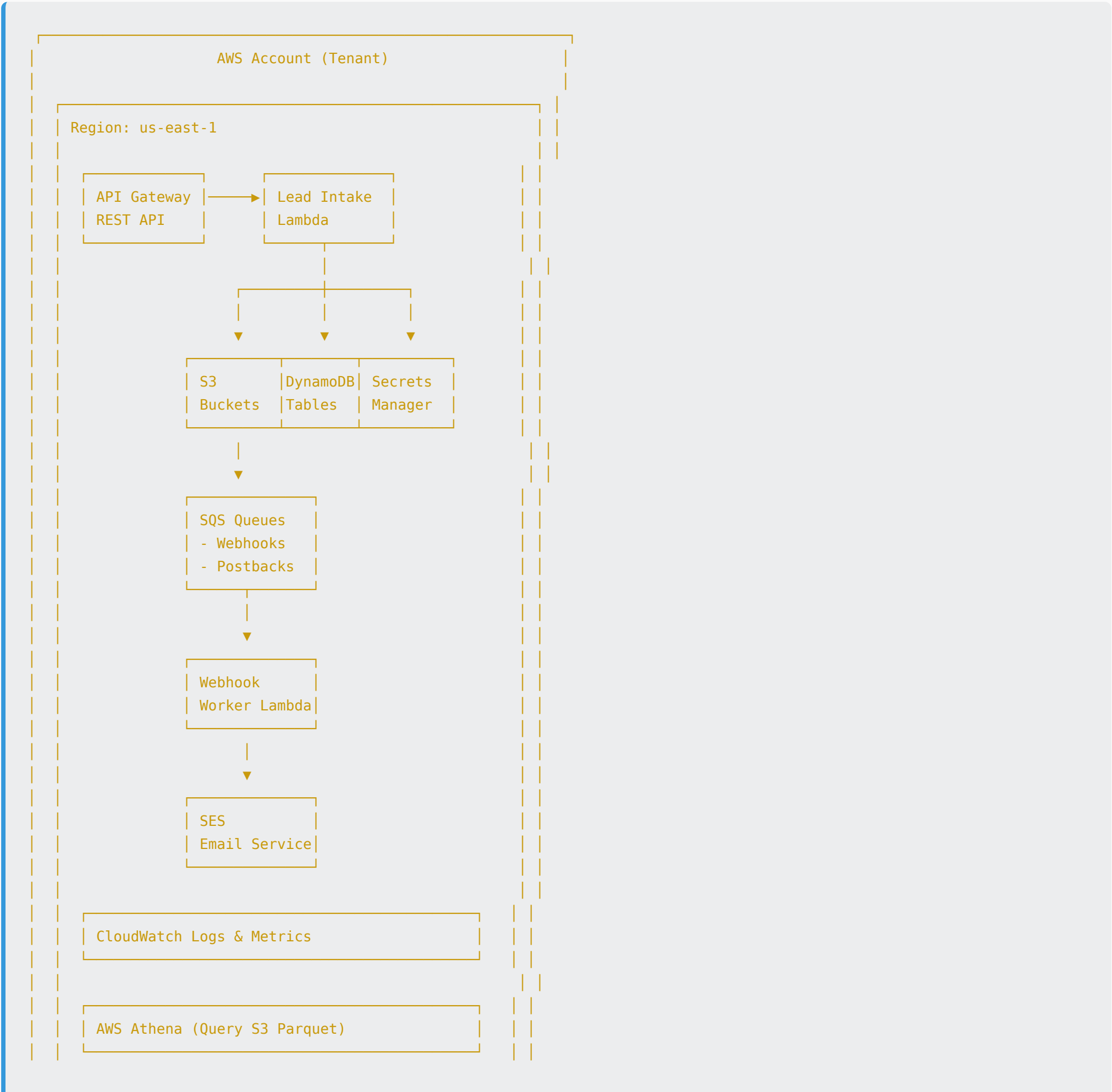
```
<!-- Similar structure with red status banner and rejection reason details -->
<div class="status-rejected">❌ LEAD REJECTED</div>

<h2>Rejection Reason</h2>
<div class="rejection-reason">{{rejection_reason}}</div>

<h2>Failure Details</h2>
<table>
  <!-- Show which check failed and why -->
</table>
```

AWS Infrastructure

Architecture Diagram



CDK Stack Structure

```
cdk/
├─ app.ts                # CDK App entry point
├─ stacks/
│  ├─ dataStack.ts       # DynamoDB tables + S3 buckets
│  ├─ iamStack.ts        # IAM roles and policies
│  ├─ servicesStack.ts   # Lambda functions
│  ├─ apiStack.ts        # API Gateway
│  ├─ queueStack.ts      # SQS queues
│  └─ secretsStack.ts    # Secrets Manager setup
└─ lib/
   ├─ constructs/
   │  ├─ lambdaFunction.ts # Reusable Lambda construct
   │  └─ dynamoTable.ts   # Reusable DynamoDB construct
   └─ config/
      └─ environment.ts    # Environment-specific configs
```

Resource Naming Convention

Pattern: {tenant}-{system}-{env}-{type}-{name}

Examples: - S3 Bucket: smashorbit-lms-prod-bucket-leads-raw - DynamoDB Table: smashorbit-lms-prod-table-campaigns - Lambda Function: smashorbit-lms-prod-function-lead-intake - SQS Queue: smashorbit-lms-prod-queue-webhooks - Secret: smashorbit-lms-prod-secret-ipqs

Lambda Functions

1. Lead Intake Lambda

Name: {tenant}-{system}-{env}-function-lead-intake

Runtime: Node.js 20 (ARM64)

Memory: 1024 MB

Timeout: 30 seconds

Environment Variables: - TENANT_ID - ENVIRONMENT - LEADS_TABLE_NAME - CAMPAIGNS_TABLE_NAME - AFFILIATES_TABLE_NAME - CLIENTS_TABLE_NAME - CONFIG_CATALOG_TABLE_NAME - TENANT_CONFIG_TABLE_NAME - S3_LEADS_BUCKET - WEBHOOK_QUEUE_URL - SES_FROM_EMAIL - IPQS_API_KEY (from Secrets Manager at deployment) - IPQS_BASE_URL (from Secrets Manager at deployment) - TRUSTEDFORM_USERNAME (from Secrets Manager at deployment) - TRUSTEDFORM_PASSWORD (from Secrets Manager at deployment)

Layers: - AWS SDK v3 - uuid - axios - parquet-wasm (for Parquet writing)

2. Webhook Worker Lambda

Name: {tenant}-{system}-{env}-function-webhook-worker

Runtime: Node.js 20 (ARM64)

Memory: 512 MB

Timeout: 60 seconds

Trigger: SQS Queue

Batch Size: 10 messages

Environment Variables: - LEADS_TABLE_NAME - WEBHOOK_DLQ_URL

DynamoDB Tables

All tables use **Pay-per-request billing** with **Point-in-time recovery** enabled.

Table Details

Table	Primary Key	Sort Key	GSI Count
Leads	id	timestamp	5
Campaigns	id	created_at	2
Affiliates	id	created_at	3

Table	Primary Key	Sort Key	GSI Count
Clients	id	created_at	3
Config Catalog	campaign_id	config_type	2
Tenant Config	tenant_id	config_type	0

S3 Buckets

1. Raw Leads Bucket

Name: {tenant}-{system}-{env}-bucket-leads-raw
Purpose: Store raw lead payloads in Parquet format
Lifecycle: - Transition to S3 Glacier after 90 days - Delete after 365 days (configurable)

Folder Structure:

/leads/{year}/{month}/{day}/leads-{year}-{month}-{day}.parquet

Athena Integration: External table for SQL queries

SQS Queues

1. Webhook Queue

Name: {tenant}-{system}-{env}-queue-webhooks
Visibility Timeout: 60 seconds
Message Retention: 4 days
DLQ: Enabled (max 3 receives)

2. Webhook DLQ

Name: {tenant}-{system}-{env}-queue-webhooks-dlq
Message Retention: 14 days

Secrets Manager

Secrets: - {tenant}-{system}-{env}-secret-ipqs : IPQS credentials - {tenant}-{system}-{env}-secret-trustedform : TrustedForm credentials

Rotation: Manual (future: automatic rotation)

Migration Strategy

Phase 1: Data Migration (Week 1-2)

Step 1: Create New Infrastructure

- Deploy v2.0 CDK stacks (data, IAM, services, API, queues)
- Set up DynamoDB tables with GSI indexes
- Create S3 buckets for Parquet storage
- Configure Secrets Manager for plugin credentials

Step 2: Migrate Existing Leads

```
// Migration script
async function migrateLeadsFromV1() {
  // 1. Scan v1 Leads table
  const v1Leads = await scanV1LeadsTable();

  // 2. Transform to v2 schema
  for (const v1Lead of v1Leads) {
    const v2Lead = {
      id: v1Lead.id,
      timestamp: v1Lead.timestamp,
```

```
// Add new fields
campaign_id: 'CAMP00000001', // Default campaign
affiliate_id: null,
client_id: null,

// Map existing fields
first_name: v1Lead.first_name,
last_name: v1Lead.last_name,
email: v1Lead.email,
phone: v1Lead.phone,

// Set defaults for new fields
remapped: false,
quality_checks: {
  ipqs: {
    enabled: true,
    passed: v1Lead.passed_phone_check && v1Lead.passed_email_check && v1Lead.passed_ip_check,
    results: v1Lead.ipqs_response || {}
  },
  trustedform: {
    enabled: true,
    passed: v1Lead.passed_tf_check,
    results: v1Lead.trustedform_response || {}
  },
  duplicate: {
    enabled: false,
    passed: true,
    duplicate_found: false
  }
},

sellable: v1Lead.sellable,
sold: v1Lead.sold,

test_mode: false,
tenant_id: process.env.TENANT_ID
};

// 3. Write to v2 Leads table
await v2LeadsTable.put(v2Lead);

// 4. Create raw payload in S3 Parquet
await writeToParquet(v2Lead);
}
}
```

Step 3: Create Default Campaign

- Create campaign with ID `CAMP00000001`
- Set base criteria matching v1 validation rules
- Set base logic matching v1 acceptance rules
- Add IPQS and TrustedForm plugins with v1 thresholds

Phase 2: Affiliate & Client Setup (Week 3-4)

Step 1: Create Affiliates

- Import existing affiliate data (if any)
- Assign unique IDs (`AFF12345678`)
- Set up postback URLs

Step 2: Create Clients

- Import existing client data (if any)
- Assign unique IDs (`CLI12345678`)
- Set up webhook URLs

- Configure routing rules

Step 3: Link to Campaign

- Add affiliates to default campaign
- Add clients to default campaign
- Test end-to-end flow with test leads

Phase 3: Parallel Run (Week 5-6)

Dual-Write Strategy

```
async function parallelRun(leadPayload) {
  // Write to both v1 and v2
  const [v1Result, v2Result] = await Promise.all([
    processLeadV1(leadPayload),
    processLeadV2(leadPayload)
  ]);

  // Compare results
  if (v1Result.result !== v2Result.result) {
    console.warn('Result mismatch detected', {
      v1: v1Result,
      v2: v2Result
    });

    // Alert for manual review
    await sendAlert('Migration validation failed');
  }

  // Return v1 result to maintain current behavior
  return v1Result;
}
```

Validation

- Compare lead processing results
- Verify quality check outcomes
- Monitor error rates
- Validate webhook deliveries

Phase 4: Cutover (Week 7)

Step 1: API Switch

- Update API Gateway to route to v2 Lambda
- Keep v1 Lambda available for rollback
- Monitor CloudWatch metrics

Step 2: Monitoring

- Track lead processing times
- Monitor DynamoDB read/write capacity
- Check S3 Parquet write performance
- Verify webhook delivery success rates

Step 3: Decommission v1

- After 2 weeks of stable v2 operation
- Archive v1 data to S3
- Delete v1 DynamoDB tables
- Remove v1 Lambda functions

Rollback Plan

Triggers: - Error rate > 5% - Lead processing time > 5 seconds (p95) - Webhook delivery rate < 95%

Steps: 1. Switch API Gateway back to v1 Lambda 2. Investigate v2 issues 3. Fix and re-deploy 4. Resume parallel run for validation

Appendices

A. Unique ID Generation

Format: 8 characters, uppercase letters and numbers, prefixed by entity type

Implementation:

```
function generateUniqueId(entityType) {
  const prefixes = {
    campaign: 'CAMP',
    affiliate: 'AFF',
    client: 'CLI',
    lead: 'LEAD'
  };

  const prefix = prefixes[entityType];
  const chars = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789';
  let id = '';

  for (let i = 0; i < 8; i++) {
    id += chars.charAt(Math.floor(Math.random() * chars.length));
  }

  return `${prefix}${id}`;
}

// Examples:
// CAMP12A4B6C8
// AFF7X9Y2Z5Q3
// CLI4R8T1U3V7
// LEAD9W2E5R8T
```

Collision Handling: - Check DynamoDB before creating entity - Retry with new ID if collision detected - Maximum 10 retry attempts

B. Testing Configuration Examples

Bypass All Checks:

```
{
  "test_config": {
    "bypass_quality_checks": true,
    "bypass_criteria": true,
    "bypass_duplicate_check": true,
    "skip_webhooks": true,
    "skip_emails": true
  }
}
```

Bypass Only Duplicate Check (for testing with same test data):

```
{
  "test_config": {
    "bypass_quality_checks": false,
    "bypass_criteria": false,
    "bypass_duplicate_check": true,
    "skip_webhooks": false,
    "skip_emails": true
  }
}
```

```
}  
}
```

C. Front-End Integration Considerations

Admin Dashboard Features: - Campaign CRUD with modal UI - Affiliate/Client management - Criteria question builder (drag-and-drop form builder) - Logic rule builder (visual workflow editor) - Plugin configuration UI - Real-time lead monitoring - Analytics dashboard (leads by campaign, affiliate performance, client stats) - S3 raw payload viewer (query Athena, display Parquet data)

API Endpoints Needed: - `POST /v2/campaigns` - Create campaign - `PUT /v2/campaigns/{id}` - Update campaign - `GET /v2/campaigns` - List campaigns - `POST /v2/campaigns/{id}/affiliates` - Add affiliate - `POST /v2/campaigns/{id}/clients` - Add client - `PUT /v2/campaigns/{id}/criteria` - Update base criteria (creates new version) - `PUT /v2/campaigns/{id}/logic` - Update base logic (creates new version) - `GET /v2/leads?campaign_id={id}` - List leads for campaign - `GET /v2/analytics/dashboard` - Get dashboard metrics

D. Performance Estimates

Lead Processing Latency: - S3 Parquet Write: ~100ms - DynamoDB Lookups: ~50ms - Field Mapping: ~10ms - IPQS API Call: ~500ms - TrustedForm API Call: ~500ms - DynamoDB Write: ~50ms - SQS Queue: ~20ms - **Total:** ~1.23 seconds (synchronous portion)

Note: Using environment variables instead of Secrets Manager API calls saves ~100ms per lead (2 secret retrievals × 50ms each)

Webhook Delivery: - Asynchronous via SQS - Does not impact lead processing latency - Average delivery time: 2-5 seconds

Throughput Capacity: - Lambda concurrency: 1000 concurrent executions - DynamoDB: Unlimited with pay-per-request - S3: Unlimited - **Estimated:** 800+ leads per second

E. Cost Estimation (Monthly)

Assumptions: - 100,000 leads per month - Average lead size: 5KB

AWS Costs: - API Gateway: \$3.50 (1M requests @ \$3.50/M) - Lambda (Lead Intake): \$8.00 (100K invocations × 1s × 1024MB) - Lambda (Webhook Worker): \$4.00 (100K invocations × 0.5s × 512MB) - DynamoDB: - Writes: \$12.50 (100K writes @ \$1.25/M) - Reads: \$2.50 (20K reads @ \$0.25/M) - Storage: \$5.00 (20GB @ \$0.25/GB) - S3: - Storage: \$0.50 (20GB Parquet @ \$0.023/GB) - Requests: \$0.50 (100K PUTs) - SQS: \$0.40 (100K messages), no API call charges) - **Total:** ~\$39/month + external API costs (IPQS, TrustedForm)

Note: By using environment variables instead of runtime Secrets Manager API calls, we save ~\$4/month in API retrieval costs (200K retrievals @ \$0.05/10K = \$1 per secret × 2 secrets × 2 = \$4 - Secrets Manager: \$0.80 (2 secrets @ \$0.40/secret) - **Total:** ~\$39/month + external API costs (IPQS, TrustedForm)

Conclusion

This architecture document provides a comprehensive blueprint for transforming the prototype lead intake system into a production-ready, multi-tenant SaaS platform. The design prioritizes:

- Data Integrity:** Raw payload storage in S3 Parquet with transformation audit trail
- Flexibility:** Campaign-based configuration with versioning and custom rules
- Scalability:** Serverless architecture supporting 800+ leads per second
- Extensibility:** Plugin-based quality checks for easy integration of new services
- Reliability:** Asynchronous webhook delivery with retry mechanisms

Next Steps: 1. Review and approve architecture 2. Create detailed implementation plan with sprint breakdown 3. Set up development environment 4. Begin Phase 1 (Data Migration) implementation

Document History: - v1.0 - February 4, 2026 - Initial architecture document